

Languages and Machines

David M. Clark

Contents

Acknowledgements	i
Introduction To The Student	ii
Introduction To The Instructor	iii
1 Discrete Mathematics	1
1.1 Sets	1
1.2 Relations, Functions, and Equivalence Relations	4
2 Finite State Machines	8
2.1 Finite State Transducers	8
2.2 Finite State Acceptors	10
3 Languages	13
3.1 Alphabets and Languages	13
3.2 Formal Grammars	15
4 Equivalences	18
4.1 FSAs Into Regular Expressions	18
4.2 Regular Expressions Into Regular Grammars	21
4.3 Regular Grammars Into NDFSAs	24
4.4 NDFSAs into (D)FSAs	27
5 Machine Minimization	31
5.1 Minimal Finite State Acceptors	31
5.2 Languages That Are Not Regular	35
6 PDAs and CFGs	37
6.1 Context-Free Languages	37
6.2 Pushdown Acceptors	39
6.3 Epilogue	41
Bibliography	45

Acknowledgements

This guide was written under the auspices of SUNY New Paltz. The author gratefully acknowledges Harry Lucas and the Educational Advancement Foundation for their support, and Allison Vandenbrul for her hard work in the preparation of the included graphics. He also wishes to acknowledge the hard work of the many SUNY New Paltz students whose efforts and feedback have led to the refined guide that is now before you.

David M. Clark
Department of Computer Science
SUNY New Paltz
New Paltz, New York 12561

Introduction To The Student

In this course, we will learn about some of the fundamental relationships between computing machines and the languages that they understand. Experience shows us that there is a fundamental tradeoff between hardware and software: the same performance can often be achieved by a complex computer running a simple (high-level) program, as by a simple computer running a complex (low-level) program. In order to understand the capabilities of different computing machines, it is helpful to overlook the hardware/software dichotomy and view the combination as a single entity. For this reason, we will use the word *machine* to refer to the combination of a computer and the software that runs it. We will formalize this vague concept in several ways as we go along.

Computer chips are now built into many different familiar devices, ranging from toasters and soda machines to high-level computers. While each of these is a machine in our general sense, some are much more complex and powerful than others. Our goal in this course is to find ways to classify machines according to their complexity, and to understand the capabilities of machines in each complexity category.

It turns out that the complexity of a particular machine is best understood by looking at how and what it is able to communicate with a user. What kind of instructions is it able to understand, and what kind of responses is it able to give? In short, we need to know what language it speaks. Simple machines communicate simple ideas in simple languages; complex machines communicate complex ideas in complex languages. We will find that there is a significant spectrum from simple to complex. This spectrum is understood by studying, simultaneously, *languages* and the *machines* that speak them.

Before we begin a study of languages and machines, there are a few preliminary tools from discrete mathematics that we need to review. Although the ideas we study here may be somewhat familiar, you may not have yet seen them laid out in the systematic and more formal way that they are needed for our purposes.

Introduction To The Instructor

This discovery guide is primarily written for advanced undergraduate computer science and computer engineering students. These students often are very capable but have never before been asked to explain in clear English why what they are doing is correct and why what they are saying is true. I have found this course to be extremely valuable for these students. I have also had them return to tell me how much they appreciated the kind of thinking it taught them, as they moved through subsequent courses.

As these students normally have no prior theorem-proving experience, I have viewed “discovery” essentially as problem-solving. Students solve and present in class all of the problems in the text, and then write up a final version of each one in a cumulative portfolio. Important results are stated throughout as theorems. As the nature of most of these theorems is to assert that there is an algorithm to do something, I have simply presented as a “proof” a description of the algorithm, and have found my students amply challenged by working through several problems that illustrate that algorithm.

Chapter 1 is written to be brief and yet to contain all of the discrete mathematics that will be needed in subsequent chapters. Throughout, languages are treated as sets of strings, thereby requiring the formalism of set theory. The definitions of all finite state machines require transition functions, which in turn require direct products to form their domains. Equivalence relations are entirely new to most of my students, yet they are required in Chapter 5 to construct minimal automata. It is essential to spend the necessary time in Chapter 1 to master these topics.

Chapters 2 through 5 give a thorough treatment of finite state automata and regular languages, and Chapter 6 offers an introduction to pushdown automata and context-free languages. I have only been able to do Chapter 6 when I have taught students who had a good prerequisite discrete mathematics course, including equivalence relations, so that I could begin in Chapter 2.

There is a recurrent theme of pedagogical significance that is sprinkled throughout the text. The last problem in Chapter 2 presents the classical example $L_{=}$ of a context-free language that is not regular. Here I ask the students if they can find a finite state acceptor (FSA) for $L_{=}$. The answer is “no”, but getting this answer provides an excellent opportunity to discuss the distinction between the assertion that “No one in this room knows of a FSA for $L_{=}$.” and “There is no FSA for $L_{=}$.” Urging them to try to find one can lead to interesting discoveries; for example, a simple “infinite state acceptor” for $L_{=}$. This observation can lead to the discovery that there is in fact an infinite state acceptor for every language, so that the existence of such a device tells us nothing at all about a language. If the students conjecture that there is no FSA for $L_{=}$, I urge them to look for a convincing argument for this. The last problem in Chapter 3 asks for a formal grammar for $L_{=}$, which can be constructed with only two simple but non-regular productions.

I let this problem hang for most of the term, perhaps mentioning it occasionally. In Chapter 5, we are finally able to use equivalence relations to prove

something interesting about $L_=$, in Problem 5.3. This can be a convincing demonstration of the relevance of equivalence relations. As an immediate consequence of Problem 5.3 and the Myhill-Nerode Corollary, we finally see that $L_=$ is not a regular language. Tracing the essence of this argument, we can give a simple bare-hands argument that $L_=$ is not regular in Example 1 without using any of this new machinery. The language $L_=$ recurs in Chapter 6, Examples 1 and 4, and our prototype context-free language. I believe that the saga of $L_=$ provides an fine example of the process of mathematical inquiry and discovery.

The correlation between languages and machines presented here is a beautiful topic which many people can appreciate. These notes are thorough and complete, and should provide adequate guidance for any instructor with a sound grasp of abstract algebra and a working sense of algorithmic computation.

I would certainly appreciate any corrections, comments, or ideas that you care to share.

David Clark
October 2002

1 Discrete Mathematics

In this chapter, we will study four basic concepts from discrete mathematics that are used throughout theoretical computer science: sets, functions, relations, and equivalence relations.

1.1 Sets

By a **set**, we mean a collection of objects taken from some fixed universe of objects that we will call **U**. If **A** is a set, then for each object x in **U**, either

$$x \in \mathbf{A} \text{ (read “} x \text{ is a member of } \mathbf{A} \text{”)}$$

or

$$x \notin \mathbf{A} \text{ (read “} x \text{ is not a member of } \mathbf{A} \text{”)}$$

If **A** and **B** are sets,

$$\mathbf{A} \subseteq \mathbf{B} \text{ (read “} \mathbf{A} \text{ is a subset of } \mathbf{B} \text{”)}$$

if each member of **A** is also a member of **B**.

Sets are assumed to satisfy two important principles that we will use frequently. In mathematics, sets are completely determined by their members:

Set Equality Principle: If **A** and **B** are sets that have the same members (that is, $\mathbf{A} \subseteq \mathbf{B}$ and $\mathbf{B} \subseteq \mathbf{A}$), then $\mathbf{A} = \mathbf{B}$ are the same set.

This notion of “set” does not always match the colloquial notion. For example, it might sometimes be the case that the students enrolled in Languages & Machines are exactly the same students as are enrolled in Calculus 2. While this would be two different classes, it would be a single set of students.

Set Specification Principle: If **P** is a property that each object in **U** either has or does not have, then there is a set denoted by

$$\{x \mid x \in \mathbf{U} \text{ and } x \text{ has property } \mathbf{P}\}$$

whose members are exactly those objects in **U** having property **P**.

For example,

$$\mathbf{S} = \{(x, y) \mid x \text{ and } y \text{ are real numbers, and } 2x + y^2 = 7\}$$

is the set of points on a parabola in the plane.

Using these two simple axioms, we can build many special sets and prove a variety of interesting facts about them. For example, our original “universe”

$$\mathbf{U} = \{x \mid x \in \mathbf{U} \text{ and } x = x\}$$

is a set, the *universal set*, where we have taken the property **P** to be $x = x$, which is true of every x in **U**. At the other extreme,

$$\emptyset = \{x \mid x \in \mathbf{U} \text{ and } x \neq x\}$$

is called the *empty set* since it has no members.

We can construct new sets from old using the Set Specification Principle:

$$\mathbf{A} \cap \mathbf{B} = \{x \mid x \in \mathbf{A} \text{ and } x \in \mathbf{B}\}$$

is called the *intersection* of **A** and **B**;

$$\mathbf{A} \cup \mathbf{B} = \{x \mid x \in \mathbf{A} \text{ or } x \in \mathbf{B}\}$$

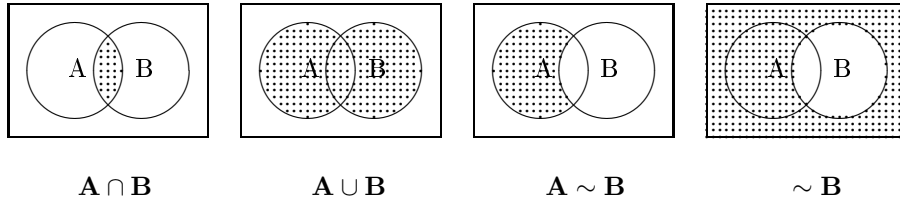
is called the *union* of **A** and **B**;

$$\mathbf{A} \sim \mathbf{B} = \{x \mid x \in \mathbf{A} \text{ but } x \text{ is not in } \mathbf{B}\}$$

is called the *difference* between **A** and **B**;

$$\sim \mathbf{B} = \mathbf{U} \sim \mathbf{B} = \{x \mid x \in \mathbf{U} \text{ but } x \text{ is not in } \mathbf{B}\}$$

is called the *complement* of **B** in **U**.



Often it turns out that different expressions represent the same set. For example, we can use the above definitions to prove the distributive property of intersection over union:

Theorem: For every choice of sets **A**, **B**, and **C**,

$$\mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$$

Proof: Let $x \in \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C})$. Then $x \in \mathbf{A}$, and either $x \in \mathbf{B}$ or $x \in \mathbf{C}$.

Case 1: $x \in \mathbf{B}$. Then $x \in \mathbf{A} \cap \mathbf{B}$, and therefore $x \in (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$.

Case 2: $x \in \mathbf{C}$. Then $x \in \mathbf{A} \cap \mathbf{C}$, and therefore $x \in (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$.

In both cases, we conclude that $x \in (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$.

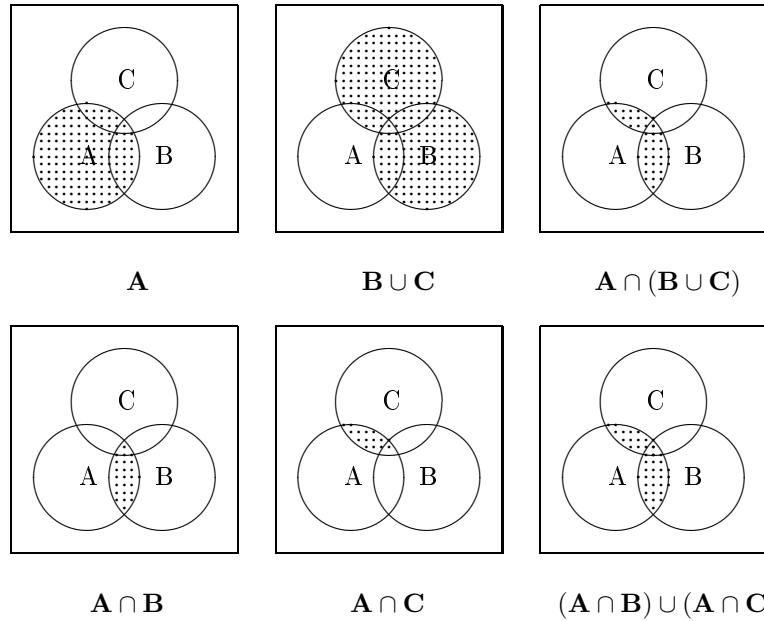
Let $x \in (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$. Then either $x \in \mathbf{A} \cap \mathbf{B}$ or $x \in \mathbf{A} \cap \mathbf{C}$.

Case 1: $x \in \mathbf{A} \cap \mathbf{B}$. Then $x \in \mathbf{A}$, and $x \in \mathbf{B} \cup \mathbf{C}$ since $x \in \mathbf{B}$. Thus $x \in \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C})$.

Case 2: $x \in \mathbf{A} \cap \mathbf{C}$. Then $x \in \mathbf{A}$, and $x \in \mathbf{B} \cup \mathbf{C}$ since $x \in \mathbf{C}$. Thus $x \in \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C})$.

In both cases, we conclude that $x \in \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C})$.

By the Set Equality Principle, $\mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})$.



Since the above theorem does not involve more than three sets, we are able to use *Venn Diagrams* to illustrate it. If $\mathbf{A}$ and $\mathbf{B}$ are sets, the *Cartesian product* of $\mathbf{A}$ and $\mathbf{B}$, denoted by $\mathbf{A} \times \mathbf{B}$, consists of all ordered pairs (x, y) where $x \in \mathbf{A}$ and $y \in \mathbf{B}$:

$$\mathbf{A} \times \mathbf{B} = \{(x, y) \mid x \in \mathbf{A} \text{ and } y \in \mathbf{B}\}$$

For any set $\mathbf{A}$, the *power set* of $\mathbf{A}$, denoted by $\mathcal{P}(\mathbf{A})$, is the set of all subsets of $\mathbf{A}$:

$$\mathcal{P}(\mathbf{A}) = \{\mathbf{B} \mid \mathbf{B} \subseteq \mathbf{A}\}$$

We sometimes write $\mathbf{B} \subsetneq \mathbf{A}$, read “ $\mathbf{B}$ is a *proper subset* of $\mathbf{A}$ ”, to mean that $\mathbf{B}$ is a subset of $\mathbf{A}$ but $\mathbf{B}$ is not equal to $\mathbf{A}$.

Problems

Confirm each of the identities in (1.1), (1.2), and (1.3) with a Venn Diagram, by drawing each side and checking that they are the same:

(1.1) the Associative Laws:

$$\begin{aligned}\mathbf{A} \cap (\mathbf{B} \cap \mathbf{C}) &= (\mathbf{A} \cap \mathbf{B}) \cap \mathbf{C} \\ \mathbf{A} \cup (\mathbf{B} \cup \mathbf{C}) &= (\mathbf{A} \cup \mathbf{B}) \cup \mathbf{C}\end{aligned}$$

(1.2) the Distributive Laws:

$$\begin{aligned}\mathbf{A} \cup (\mathbf{B} \cap \mathbf{C}) &= (\mathbf{A} \cup \mathbf{B}) \cap (\mathbf{A} \cup \mathbf{C}) \\ \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) &= (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C})\end{aligned}$$

(1.3) DeMorgan's Laws:

$$\begin{aligned}\sim (\mathbf{A} \cap \mathbf{B}) &= \sim \mathbf{A} \cup \sim \mathbf{B} \\ \sim (\mathbf{A} \cup \mathbf{B}) &= \sim \mathbf{A} \cap \sim \mathbf{B}\end{aligned}$$

(1.4) Which one of the following three statements is always true? Find examples of sets $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ for which each of the remaining two fail.

- $\mathbf{A} \cap (\mathbf{B} \times \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \times (\mathbf{A} \cap \mathbf{C})$
- $\mathbf{A} \times (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \times \mathbf{B}) \cup (\mathbf{A} \times \mathbf{C})$
- $\mathbf{A} \cup (\mathbf{B} \times \mathbf{C}) = (\mathbf{A} \cup \mathbf{B}) \times (\mathbf{A} \cup \mathbf{C})$

(1.5) If $\mathbf{A}$ has 5 elements, how many elements does $\mathcal{P}(\mathbf{A})$ have? If $\mathbf{A}$ has n elements, how many elements does $\mathcal{P}(\mathbf{A})$ have?

1.2 Relations, Functions, and Equivalence Relations

A *relation* from a set $\mathbf{A}$ to a set $\mathbf{B}$ is a subset r of $\mathbf{A} \times \mathbf{B}$. We call $\mathbf{A}$ the *domain* of r , and call $\mathbf{B}$ the *codomain* of r . If $\mathbf{A} = \mathbf{B}$, we say that r is a *relation on* $\mathbf{A}$. For $x \in \mathbf{A}$ and $y \in \mathbf{B}$, we often write “ xry ” to mean that $(x, y) \in r$. For example, the “less than or equal to” relation $\leq$ on the set $\mathbf{R}$ of real numbers is formally defined as

$$\leq = \{(x, y) \mid x \text{ is less than or equal to } y\}$$

We normally write “ $3 \leq 7$ ” instead of “ $(3, 7) \in \leq$ ”.

One special kind of relation is a *function*. A function f from $\mathbf{A}$ to $\mathbf{B}$ is a rule which associates each member $x \in \mathbf{A}$ with a member $y = f(x) \in \mathbf{B}$. We express this by writing

$$f : \mathbf{A} \rightarrow \mathbf{B}$$

and saying that “ f is a function from $\mathbf{A}$ to $\mathbf{B}$ ”. The *range* of f , denoted by $f(\mathbf{A})$, is the subset $f(\mathbf{A}) = \{f(x) \mid x \in \mathbf{A}\}$ of $\mathbf{B}$ consisting of all *images* $f(x)$ of elements x in $\mathbf{A}$. If $\mathbf{A}$ is a finite set, then it is often convenient to store a function f from $\mathbf{A}$ to $\mathbf{B}$ as the set of ordered pairs (x, y) , where $x \in \mathbf{A}$ and $y = f(x) \in \mathbf{B}$. Thus we can define a **function** f as *being* a relation from $\mathbf{A}$

to $\mathbf{B}$ in which each member of $\mathbf{A}$ occurs exactly once as the first term of some ordered pair. This gives us

$$f = \{(x, y) \mid x \in \mathbf{A} \text{ and } y = f(x) \in \mathbf{B}\} \subseteq \mathbf{A} \times \mathbf{B}$$

We say that f is

- *one-to-one* if $f(x) \neq f(y)$ whenever $x, y \in \mathbf{A}$ and $x \neq y$;
- *onto* $\mathbf{B}$ if for every $y \in \mathbf{B}$, there is an $x \in \mathbf{A}$ such that $f(x) = y$.

We call f a *bijection* if it is both one-to-one and onto $\mathbf{B}$. We say that $\mathbf{A}$ and $\mathbf{B}$ are the *same size* if there exists some bijection from $\mathbf{A}$ onto $\mathbf{B}$.

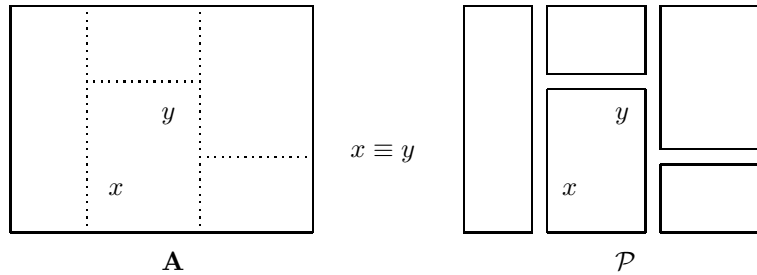
Another special kind of relation is an *equivalence relation*, an idea which requires some digression to explain. There are many occasions in computer science when we want to describe certain objects as being “equivalent” in some sense. For example, two programs might be thought of as “equivalent” if they always produce the same output from the same input. Of course, this notion is not clear until we specify what inputs are allowable. If we allow a larger set of possible inputs, we can expect fewer programs to be “equivalent”. When we clean up or simplify a program, we are producing a new program which is equivalent to the old one, but is in some way easier to read. Alternatively, we might fix a program and fix a set of possible input strings, and define two input strings to be “equivalent” if they generate the same output from the program. This would give us a different notion of “equivalence” of input strings for each program. In a very different context, we might think of two formulas in propositional logic to be “equivalent” if they have the same truth tables.

All of these notions of “equivalent” have something in common. In each case, there is a set (programs, input strings, formulas) which is broken up into disjoint bunches so that two members are “equivalent” exactly when they are in the same bunch. This observation leads to the following definition. We say that a collection $\mathcal{P}$ of non-empty subsets of a set $\mathbf{A}$ is a *partition* of $\mathbf{A}$ if each member of $\mathbf{A}$ is in exactly one member of $\mathcal{P}$; that is,

1. $\bigcup \mathcal{P} = \mathbf{A}$ (the union of all the sets in $\mathcal{P}$); and
2. $X \cap Y = \emptyset$ for every pair of distinct $X, Y \in \mathcal{P}$. (We say X and Y are *disjoint*.)

Associated with any partition $\mathcal{P}$ of $\mathbf{A}$ is a relation $\equiv_{\mathcal{P}}$ of “equivalence” on $\mathbf{A}$: Elements x and y of $\mathbf{A}$ are “equivalent”; that is, $x \equiv_{\mathcal{P}} y$, if x and y are in the same member of $\mathcal{P}$. A relation $\equiv_{\mathcal{P}}$ that arises out of a partition $\mathcal{P}$ in this way always has three special properties:

- $\equiv$ is *reflexive*: $x \equiv x$ for any $x \in \mathbf{A}$;
- $\equiv$ is *symmetric*: $y \equiv x$ whenever $x \equiv y$;
- $\equiv$ is *transitive*: $x \equiv z$ whenever $x \equiv y$ and $y \equiv z$.



A binary relation $\equiv$ which has these three properties is called an *equivalence relation*. Thus $\equiv_{\mathbf{P}}$ is an equivalence relation for every partition $\mathbf{P}$. What is surprising is that every equivalence relation can be obtained in this way from some partition.

Equivalence Relation Theorem: Let $\equiv$ be an equivalence relation on the set $\mathbf{A}$. For each $x \in \mathbf{A}$, define the $\equiv$ -class of x to be the set

$$[x] = \{y \mid y \in \mathbf{A} \text{ and } x \equiv y\}$$

Then the set $\mathcal{P} = \{[x] \mid x \in \mathbf{A}\}$ of all $\equiv$ -classes forms a partition of the set $\mathbf{A}$ such that $x \equiv y$ if and only if x and y are in the same member of $\mathcal{P}$.

Proof: Every member x of $\mathbf{A}$ is in some $\equiv$ -class, namely the class $[x]$ that it determines. This is because $\equiv$ is *reflexive*, so $x \equiv x$. This tells us that $\bigcup \mathcal{P} = \mathbf{A}$.

To see that distinct $\equiv$ -classes are disjoint, suppose that $[x] \cap [y] \neq \emptyset$. We will show that $[x] = [y]$. Let $u \in [x]$; that is, $x \equiv u$. Choose any $z \in [x] \cap [y]$. Since $x \equiv z$, we have, by *symmetry*, $z \equiv x$. Since $y \equiv z \equiv x \equiv u$, we have, by *transitivity*, $y \equiv u$. Thus $u \in [y]$. Similarly, $[y] \subseteq [x]$, so $[x] = [y]$. This shows that the $\equiv$ -classes $[x]$ and $[y]$ are either equal or disjoint.

Problems

In (1.6) through (1.10), verify that the relation is an equivalence relation, and then describe its equivalence classes.

(1.6) $\mathbf{A}$ is the set of points (x, y) in the plane. $(x, y) \equiv (x', y')$ if (x, y) and (x', y') are the same distance from the origin.

(1.7) $x \equiv y$ if x and y are integers and $x - y$ is a multiple of 3.

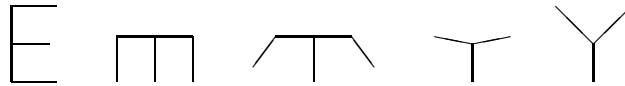
(1.8) $x \equiv y$ if x and y are integers and $x + y$ is even.

(1.9) Let $\mathbf{S} = \{a, b, c, d, e, f, g, h\}$, and let $\mathbf{T} = \{a, b, c\}$. For $X, Y \in \mathcal{P}(\mathbf{S}) = \mathbf{A}$, we define $X \equiv Y$ if X and Y have the same intersection with $\mathbf{T}$.

- (1.10) Let f be any function from the set $\mathbf{A}$ into a set $\mathbf{B}$. For x and y in $\mathbf{A}$, we say that x is *equivalent to y modulo f* if $f(x) = f(y)$.
- (1.11) Let $\mathbf{A} = \{0, 1\} \times \{0, 1\} \times \{0, 1\} \times \{0, 1\} \times \{0, 1\} = \{0, 1\}^5$ be the set of 32 different five-tuples of 0s and 1s. For $x, y \in \mathbf{A}$, define $x \equiv y$ to mean that x and y have the same number of 1s. Write down all the members of each of the different $\equiv$ -classes.
- (1.12) Suppose $\mathbf{A} = \{0, 1\}^n$ is the set of all n -tuples of 0s and 1s. For $x, y \in \mathbf{A}$, again define $x \equiv y$ to mean that x and y have the same number of 1s. How many different $\equiv$ -classes does $\mathbf{A}$ have?
- (1.13) Computer graphics uses techniques to continuously transform one image into another. As an example of this, consider the letters of the alphabet:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Two letters are said to be *topologically equivalent* if each can be continuously transformed into the other by bending, stretching, shrinking and rotating. Cutting and gluing are not allowed; nor can a point be stretched into a segment, or a segment shrunk to a point. For example, E is equivalent to Y:



How many different equivalence classes of letters are there? List the members of each.

2 Finite State Machines

In this chapter, we will study the simplest class of computing machines. These machines are important for two reasons.

- They model the logic of familiar machines – such as vending machines, washing machines, and toasters – which require logic circuitry but are not normally thought of as computers.
- They are simple enough to completely analyze from several different perspectives, illustrating how we would optimally like to understand more complex machines.

Finite state machines are characterized by the fact that their action, at any point in their operation, is determined only by their current state and their current input. They have no long-term memory, and are therefore unable to carry out any logical process of reviewing the past or anticipating the future. The reader might well picture some biological models of this sort of behavior.

2.1 Finite State Transducers

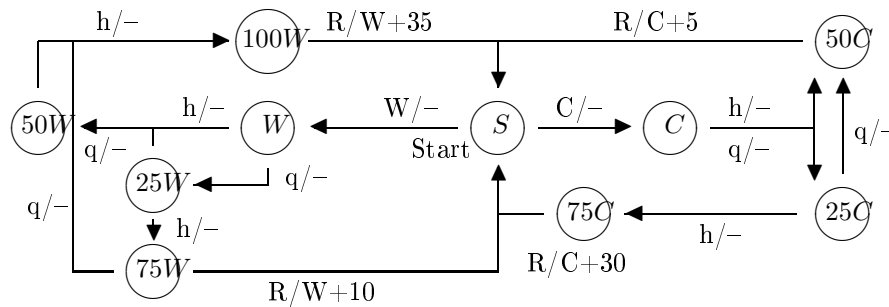
Before giving any general definitions, let's look at an example of a simple vending machine. The directions on the machine ask the user to follow three steps.

1. Choose either
 - (W) Pure Spring Water (65¢) or
 - (C) Coca-Cola (45¢).
2. Insert either quarters (q) or half-dollars (h).
3. Press "RETURN" (R) for your drink and change.

In order to execute its instructions, the machine needs to be able to remember, at any time, the user's selection and the amount of money it has received. Memory will take the form of *internal states*, one for each possible data set. The machine logic is represented by a directed labeled graph. The internal states form the vertices of the graph. Each arrow tells

1. for a particular input, what the output should be; and
2. what new internal state it should go to.

The start state is "S", and each other state has a name that indicates what it is to remember. A "-" means no output.



For example, if the user asks for a Water and puts in a quarter and a half dollar and then presses “RETURN”, the machine will go from state S to W to $25W$ to $75W$, and then will output a Water and a dime.

Our vending machine is an example of a large class of finite state machines. A *finite state transducer* is a quintuple

$$M = \langle \Sigma_1, \Sigma_2, Q, s, \delta \rangle$$

where

- Σ_1 is a finite set of symbols, called the *input alphabet*;
- Σ_2 is a finite set of symbols, called the *output alphabet*;
- Q is a finite set, the *internal states* of M ;
- $s \in Q$ is called the *initial state* of M ;
- $\delta : Q \times \Sigma_1 \rightarrow Q \times \Sigma_2$ is called the *state transition function*.

A finite state transducer is illustrated as a directed labeled graph whose vertices are the states of Q , and we label the arrow

$$q \xrightarrow{a/d} q'$$

from state q to state q' with a/d if $\delta(q, a) = (q', d)$; that is, in state $q \in Q$ reading input symbol $a \in \Sigma_1$, the machine is to go to state $q' \in Q$ and output symbol $d \in \Sigma_2$.

Problems

- (2.1) Construct a transition diagram for a finite state transducer that will read a sequence of letters from the set $\{a, b, c\}$ starting from the left (for example, “caabbacccbacb”). As output, it will change each letter to an “e” until it has read three “a”s. After that, it will change each “a” to “b”, each “b” to “c”, and each “c” to “a”.
- (2.2) Construct a transition diagram for a stamp vending machine that asks the user to

1. choose either a 10-stamp book (\$3.40) or a 20-stamp book (\$6.80);
2. insert either \$1, \$5, or \$10 bills;
3. press “RETURN” for stamps and change.

You may assume that the user never inserts a bill that will need to be returned as change; for example, a \$1 followed by a \$10.

- (2.3) Construct a transition diagram for a finite state transducer which will take as input a sequence of letters from $\{a, b, c\}$, ending with a single “ x ”. It will output a blank symbol – for each a , b , and c . When it reads “ x ”, it will output the *remainder* when the number of “ a ”s in the input is divided by 3. (Thus its output will be 0, 1, or 2.)

2.2 Finite State Acceptors

In order to fully analyze the capabilities of finite state machines, we will focus on a particular variant of this concept which is designed to answer YES or NO question. A *finite state acceptor* (FSA) is a quintuple

$$M = \langle \Sigma, Q, s, Y, \delta \rangle$$

where

- Σ is a finite set of symbols, called the *input alphabet*;
- Q is a finite set, the *internal states* of M ;
- $s \in Q$ is called the *initial state* of M ;
- Y is a subset of Q called the *accepting states* of M ;
- $\delta : Q \times \Sigma \rightarrow Q$ is called the *state transition function*.

Like a finite state transducer, a finite state acceptor is illustrated by a directed labeled graph. The vertices of the graph are again the states of Q , and there is a labeled arrow for each state $q \in Q$ and each symbol $a \in \Sigma$. The arrow

$$q \xrightarrow{a} q'$$

indicates that $\delta(q, a) = q'$. In practice, this means that M in state q reading input symbol a , will go into state q' .

The function of M will be to read a string w of symbols from Σ and either accept or reject it. To explain how this works, we need a few definitions. Let Σ^* denote the set of all finite strings of symbols from Σ . It is convenient to always include the *empty string*, which we denote by λ , in Σ^* . The transition function δ can be extended recursively to a function

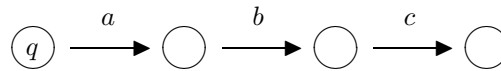
$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

by defining

1. $\delta^*(q, \lambda) = q$ and $\delta^*(q, a) = \delta(q, a)$ for $a \in \Sigma$;
2. $\delta^*(q, au) = \delta^*(\delta(q, a), u)$ for $a \in \Sigma, u \in \Sigma^*$.

For example, if $q \in Q$ and $a, b, c \in \Sigma$, then we have

$$\delta^*(q, abc) = \delta^*(\delta(q, a), bc) = \delta^*(\delta(\delta(q, a), b), c) = \delta(\delta(\delta(q, a), b), c)$$



If M begins in its start state s reading a string $w \in \Sigma^*$, then $\delta^*(s, w)$ is the state it will be in when it reaches the end of w and halts. We say that M *accepts* the string w if $\delta^*(s, w) \in Y$; otherwise, it *rejects* w . The set

$$L = \mathcal{L}(M) := \{w \in \Sigma^* \mid \delta^*(s, w) \in Y\}$$

consists of all strings accepted by M , and is called *the language recognized by M* . We think of each string in Σ^* as expressing a question (namely, “Am I in L ?”). The finite state acceptor answers YES if, reading w , it ends in a state of Y ; otherwise, it answers NO. In the transition diagram, we mark the start state with “START”, and indicate the accepting states of Y with a double circle.

In practice, we often find that after reading only part of a string, we can be certain regardless of what follows, that the answer will be NO. In these cases, it is useful to specify a “dead state” which is non-accepting and cannot be exited. We frequently denote the dead state by Z , and since many arrows go to Z , follow the convention that undrawn arrows are intended to go to Z .

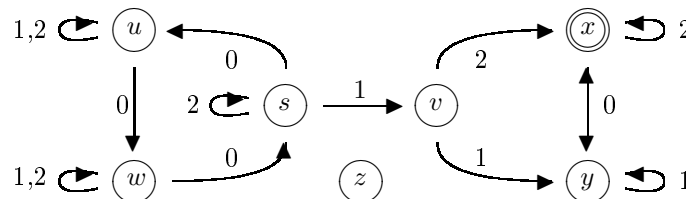
We denote the collection of languages recognizable by some FSA by

$$\mathbb{FSA} := \{L \mid L = \mathcal{L}(M) \text{ for some finite state acceptor } M\}$$

An understanding of what languages are in $\mathbb{FSA}$ will give us an indication of the capabilities of finite state machines.

Problems

- (2.4) Consider the FSA M illustrated below. Here s is the start state, x (indicated with a double circle) is the only accepting state, and all undrawn arrows go to the “dead” state z .



List three strings in $\{0, 1, 2\}^*$ that are accepted by M , and three strings that are not accepted by M .

- (2.5) Let $\Sigma = \{a, b, c, d\}$. Construct a FSA M_1 that will answer the question, “Does w begin with ab , contain exactly four c s, and end with d ?”.
- (2.6) Construct an FSA M_2 that recognizes the language consisting of all strings that are either
- a positive multiple of three 2s, followed by a possibly empty string of 0s and 1s; or
 - a positive multiple of two 3s, followed by a possibly empty string of 1s and 2s.
- (2.7) Construct an FSA that recognizes the union of the languages recognised by M_1 and M_2 in (2.5) and (2.6) above.
- (2.8) The *concatenation* of $\mathcal{L}(M_1)$ and $\mathcal{L}(M_2)$ consists of all strings uv (u followed by v), where M_1 accepts u , and M_2 accepts v . Construct an FSA that recognizes the concatenation of the languages $\mathcal{L}(M_1)$ and $\mathcal{L}(M_2)$ in (2.5) and (2.6) above.

In the following problems, assume that L_1 and L_2 are arbitrary languages in FSA, where $M_1 = \langle \Sigma, Q_1, s_1, Y_1, \delta_1 \rangle$ recognizes L_1 , and $M_2 = \langle \Sigma, Q_2, s_2, Y_2, \delta_2 \rangle$ recognizes L_2 .

- (2.9) Demonstrate that FSA is *closed under complement*; that is, if $L_1 \in \text{FSA}$, then $\Sigma^* \sim L_1 \in \text{FSA}$. (Tell how to use M_1 to design a FSA M that will recognize $\Sigma^* \sim L_1$.)
- (2.10) Demonstrate that FSA is *closed under intersection*; that is, if $L_1, L_2 \in \text{FSA}$, then $L_1 \cap L_2 \in \text{FSA}$. (Tell how to use M_1 and M_2 to design a FSA M that will recognize $L_1 \cap L_2$. Hint: Try using $Q = Q_1 \times Q_2$ for the states of M .)
- (2.11) Demonstrate that FSA is *closed under union*; that is, if $L_1, L_2 \in \text{FSA}$, then $L_1 \cup L_2 \in \text{FSA}$. (Tell how to use M_1 and M_2 to design a FSA M that will recognize $L_1 \cup L_2$. See (2.10).)

Do you think there is a FSA for $L_+ := \{a^n b^n \mid n = 1, 2, 3, \dots\}$?

3 Languages

Computing machines manipulate strings of symbols, which they take as input and give back as output. In this chapter, we will take a close look at the way symbols can be put together into strings, which in turn are put together into languages.

3.1 Alphabets and Languages

By an *alphabet* Σ , we mean a finite set of *symbols*. A *string* over alphabet Σ is a sequence of symbols from Σ written in a particular order. We denote by Σ^* the set of all strings over the alphabet Σ ; including the *empty string*, which will be represented by λ (as it is otherwise very hard to see). If $w \in \Sigma^*$ is a string, we denote the *length* of w by $|w|$. Given strings $u, v \in \Sigma^*$, the *concatenation* of u and v is the string $w = uv$ formed by tacking v onto the end of u . We say that u is a *prefix* of w , and that v is a *suffix* of w . If $x, y, z \in \Sigma^*$ and $w = xyz$, we say that y is a *substring* of w . Nonnegative integral powers of a string w are defined *recursively* by saying

1. $w^0 = \lambda$ and
2. $w^{n+1} = w^n w$

Thus $w^n \in \Sigma^*$ if $w \in \Sigma^*$.

The Roman alphabet $\Sigma = \{a, b, c, \dots, x, y, z\}$ is an example of an alphabet. The words in the dictionary are all members of Σ^* , but other strings like *quizzottlmf* are in Σ^* as well. The string *watchdog* is the concatenation of *watch* and *dog*. *watch* is a prefix of *watchdog*, but so are *wa* and *watc* as well. $|watchdog| = 8$; and in general, $|uv| = |u| + |v|$. A dog that barks too much might be called a $(watchdog)^3 = watchdogwatchdogwatchdog$. The string λ serves as an identity, since $\lambda w = w\lambda = w$ for every string w .

By a *language over* Σ , we mean a subset L of Σ^* . While every subset of Σ^* is a language, some languages are of much more interest than others. The set of all strings of Roman letters which contain exactly 13 *ps* is a language, as is the set of strings which form a grammatically correct English sentence. We will often be interested in creating new languages from old ones. If L_1 and L_2 are languages over Σ , then the language

$$L_1 L_2 := \{w \in \Sigma^* \mid w = uv \text{ for some } u \in L_1, v \in L_2\}$$

is called the *concatenation* of L_1 and L_2 . If L is a language over Σ^* , then the language

$$L^* = \{w \in \Sigma^* \mid w = u_1 u_2 \dots u_n \text{ for some } n \geq 0 \text{ and some } u_1, \dots, u_n \in L\}$$

is called the *Kleene star* of L . Note that when $n = 0$, we get $\lambda \in L^*$.

The ideas of concatenation and Kleene star give us a way to describe the languages we will study in this chapter. For an alphabet Σ , we can recursively define the set of *regular expressions* over Σ as follows.

1. $\emptyset$, λ , and each symbol $a \in \Sigma$ is a regular expression.
2. If α and β are regular expressions, then $(\alpha\beta)$ is a regular expression.
3. If α is a regular expression, then α^* is a regular expression.
4. If α and β are regular expressions, then $(\alpha \sqcup \beta)$ is a regular expression.

Thus, for example, $((cc)^*(b^*(a^* \sqcup c)^*))$ is a regular expression over $\{a, b, c\}$. A regular expression w defines a language $\mathcal{L}(w)$, which we also define recursively.

1. $\mathcal{L}(\emptyset) = \emptyset$, the empty language; $\mathcal{L}(\lambda) = \{\lambda\}$, a language containing only one string which is very short; and $\mathcal{L}(a) = \{a\}$ for each $a \in \Sigma$.
2. $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha)\mathcal{L}(\beta)$, the concatenation of $\mathcal{L}(\alpha)$ and $\mathcal{L}(\beta)$.
3. $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$, the Kleene star of $\mathcal{L}(\alpha)$.
4. $\mathcal{L}(\alpha \sqcup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$, the union of $\mathcal{L}(\alpha)$ and $\mathcal{L}(\beta)$.

Pursuing the above example, if $\alpha = ((cc)^*(b^*(a^* \sqcup c)^*))$, then

$$\begin{aligned}\mathcal{L}(\alpha) &= \mathcal{L}((cc)^*)\mathcal{L}(b^*)\mathcal{L}((a^* \sqcup c)^*) \\ &= \mathcal{L}(cc)^*\mathcal{L}(b)^*\{\mathcal{L}((a^*) \cup \mathcal{L}(c))^*\} \\ &= \{cc\}^*\{b\}^*\{\{a\}^* \cup \{c\}\}^* \\ &= \{cc\}^*\{b\}^*\{a, c\}^*\end{aligned}$$

This consists of λ , all strings of as or cs , and all strings beginning with an even number of cs followed by a string of bs followed by a (possibly empty) string of as and cs .

A language that can be represented by a regular expression is called a *regular language*. The set of all regular languages over some alphabet is denoted by

$$\text{REG} := \{\mathcal{L}(\alpha) \mid \alpha \text{ is a regular expression}\}$$

Problems

(3.1) Construct a FSA to recognize the language $\mathcal{L}(\alpha)$ where

$$\alpha := (010)^*(333)(1 \sqcup 4)^* \sqcup 0^*(3^* \sqcup 2^*)$$

(3.2) If $w \in \Sigma^*$, we define $w^R \in \Sigma^*$ recursively as follows.

1. $\lambda^R = \lambda$ and $a^R = a$ for all $a \in \Sigma$.
2. If $w = au$, where $a \in \Sigma$ and $u \in \Sigma^*$, then $w^R = u^R a$.

What string is w^R if $w = tar$? if $w = senihcamdnasegaugnal$?

(3.3) Let $\Sigma = \{a, b\}$. Give examples of two strings in L and two strings not in L where $L = \{w \in \Sigma^* \mid w^R w = w w^R\}$.

- (3.4) Let $\Sigma = \{a, b\}$. Make a complete list of all of the strings in the finite language $L = \{w \mid \text{for some } u \in \Sigma^*, w = uu^R u\}$.
- (3.5) Let $\Sigma = \{0, 1\}$. Find a regular expression representing the languages consisting of
1. all strings in Σ^* with no more than three 1s;
 2. all strings in Σ^* with a number of 1s divisible by 3;
 3. all strings in Σ^* not containing the substring 11.
- (3.6) Let $\Sigma = \{a, b, c\}$. Compute $\mathcal{L}(\alpha)$, where $\alpha = (a^* \sqcup b^* \sqcup c^*)^* b (a^* b^* c^*)^*$, and give a simple verbal description of the strings in this language.
- (3.7) Let $\Sigma = \{a, b\}$. Describe a method to generate all of the strings in the language $L = \{w \mid \text{for some } u \in \Sigma^*, wuw = uu\}$, and explain why your method is correct.

3.2 Formal Grammars

Regular expressions provide a means of specifying certain fairly simple languages. A much more powerful method of language specification is given by means of something called a “formal grammar”. The idea is to use syntactic or grammatical rules to tell how to construct the strings in the language. Although spoken languages are especially complex, we might begin to specify the grammatically correct sentences of English with rules that look something like this: Let S denote a sentence. Any sentence should have two components, a *subject* and a *predicate*. For example, in the sentence

“The noisy dog chased the sneaky cat.”

the subject is “The noisy dog” and the predicate is “chased the sneaky cat”. We might indicate this fact with the *production rule*

$$S \rightarrow JP$$

A subject J , in turn, is a *noun phrase*, typically consisting of several modifying words followed by a noun; and a predicate P is a verb followed by a noun phrase (its direct object). We could specify this by the production rules

$$J \rightarrow MN \quad P \rightarrow VJ.$$

There are many allowable modifying words, for example:

$$M \rightarrow \text{The, the, } M \text{ noisy, } M \text{ sneaky, } M \text{ overfed, } M \text{ clean, } M \text{ dirty}$$

and nouns and verbs, for example:

$$N \rightarrow \text{dog, cat, rat} \quad V \rightarrow \text{chased, complemented, ate}$$

Using these production rules, we can start with the sentence symbol S , and generate a production sequence that not only gives us a sentence in the language, but also illustrates the grammatical construction of the sentence. Here we include braces $\{ \}$, which are not symbols in the language, to record this construction:

$$\begin{aligned} S &\rightarrow JP. \rightarrow \{MN\}\{VJ\}. \\ &\rightarrow \{ \{M \text{ noisy} \} \{dog\} \} \{ \{chased\} \{MN\} \}. \\ &\rightarrow \{ \{The \text{ noisy} \} \{dog\} \} \{ \{chased\} \{ \{M \text{ sneaky} \} \{cat\} \} \}. \\ &\rightarrow \{ \{The \text{ noisy} \} \{dog\} \} \{ \{chased\} \{ \{the \text{ sneaky} \} \{cat\} \} \}. \end{aligned}$$

These ideas are quite general, as we now show. A *formal grammar* $G = \langle \Sigma, N, S, \rightarrow \rangle$ is a quadruple consisting of the following:

- a finite alphabet Σ of *terminal symbols*;
- a finite alphabet N of *nonterminal symbols*;
- a special nonterminal symbol S called the *starting symbol*;
- a relation $\rightarrow$ on $(\Sigma \cup N)^*$ called the *production rules* and written as $u \rightarrow v$ where $u, v \in (\Sigma \cup N)^*$.

The formal grammar G determines a language $\mathcal{L}(G)$ consisting of all strings w in Σ^* obtainable from a *production sequence* starting with S and using the production rules of G . Notice that the sentences in the language $\mathcal{L}(G)$ are only those strings generated from S which contain no nonterminal symbols.

As a familiar example of a grammar for a language, the set of regular expressions over a given alphabet Σ is itself a language over the larger alphabet

$$\Sigma' := \Sigma \cup \{\emptyset, \lambda, (,), +, \sqcup, \star\}$$

The definition of a “regular expression” is a specification of this language that is exactly equivalent to the grammar with production rules

$$S \rightarrow \emptyset \quad S \rightarrow \lambda \quad S \rightarrow a$$

for each $a \in \Sigma$, together with

$$S \rightarrow (SS) \quad S \rightarrow S^* \quad S \rightarrow (S \sqcup S)$$

For example, using this grammar, we can generate the regular expression

$$((33)^*(2^*(1^* \sqcup 3)^*))$$

using the following production sequence.

$$\begin{aligned} S &\rightarrow (SS) \rightarrow (S^*(SS)) \\ &\rightarrow ((SS)^*(S^*S^*)) \\ &\rightarrow ((33)^*(2^*(S \sqcup S)^*)) \\ &\rightarrow ((33)^*(2^*(S^* \sqcup 3)^*)) \\ &\rightarrow ((33)^*(2^*(1^* \sqcup 3)^*)) \end{aligned}$$

Problems

- (3.8) Use the above grammar for the language of regular expressions to construct a production sequence for the regular expression

$$(((a + c) \sqcup \lambda) ((\emptyset \sqcup b)^* ((a \sqcup (b \sqcup c))^*)))$$

- (3.9) Construct a production sequence for each of the following sentences, using the above production rules for English.

1. "The dirty rat complemented the clean cat."
2. "The overfed cat ate the noisy rat."

- (3.10) Let L_{alg} be the language consisting of all meaningful algebraic expressions, in the sense of high school algebra, over the alphabet $\Sigma = \{w, x, y, z, (,), +, -, \cdot, \div\}$. For example, " $((x + z) \cdot (w + y)) \div (w - x)$ " is in L_{alg} , but " $xy + y \div z$ " and " $(w \cdot y) \div (x - (y + z)) \div x$ " are in Σ^* but not in L_{alg} . Write down a formal grammar G for the language L_{alg} .

- (3.11) Let L_{set} be the language consisting of all expressions over the alphabet $\Sigma = \{A, B, C, \emptyset, U, (,), \cup, \cap, \sim, \times\}$ which define a new set for any choice of sets A , B , and C . For example, " $((B \cup \sim C) \times (A \cup C)) \sim (B \cap C)$ " is in L_{set} , but " $AB \cap C \times B \cup$ " and " $(C \times A) \cap (B \sim (A \cup C)) \times B$ " are in Σ^* but not in L_{set} . Write down a formal grammar G for the language L_{set} .

- (3.12) Find a formal grammar that generates $L_ = \{a^n b^n \mid n = 1, 2, 3, \dots\}$.

4 Equivalences

Languages and machines is a subject primarily concerned with language specifications. In general, there are two kinds of language specifications:

Generating Algorithm: These specifications give a method of generating all strings in the language.

Test Algorithm: These specifications give a method to test a particular string in order to tell whether or not it is in the language.

Generating Algorithm specifications are often simple and easy to come by, but they fail to give us a method to determine if a particular string is in the language. Regular expressions are examples of Generating Algorithm specifications. By making appropriate choices for each occurrence of $\star$ and $\sqcup$, we can generate every string in the language represented by a regular expression α . But in order to determine if a string w is in $\mathcal{L}(\alpha)$, you have to figure out if there is some way to use the constructions specified by α to create w .

Test Algorithm specifications give a method to find out if an arbitrary string w is in the language being specified. Such specifications may be complex, but applying the algorithm must require no decisions or judgement on the part of the user who is simply to follow the instructions. These specifications are often given by a machine which is thought of as implementing the algorithm. Finite state acceptors are typical examples of test algorithm specifications of languages.

In this chapter, we will look at some relationships between these two forms of specifications.

4.1 FSAs Into Regular Expressions

In Problem (2.4), you used ad-hoc methods to find a regular expression for the language recognized by a particular FSA. It turns out that there is a very nice method to find a regular expression for the language specified by any FSA. In this section, we will present this method.

Look again at the FSA M of problem (2.4). Suppose that we use the name of each state to represent the *set* of all strings that lead from that state to an accepting state. In particular, s represents the language $\mathcal{L}(M)$. Looking at the diagram, we can write down equations that relate these sets. Using “+” to represent “union”, we have for example,

$$s = 2s + 0u + 1v$$

This equation says that the strings leading from s to an accepting state are exactly those consisting of a 2 followed by another one of those, those consisting of a 0 followed by a string leading from u to an accepting state, and those consisting of a 1 followed by a string leading from v to an accepting state. We

can write down a similar equation for each of the other five states:

$$\begin{aligned} u &= \{1, 2\}u + 0w \\ w &= \{1, 2\}w + 0s \\ v &= 2x + 1y \\ x &= 2x + 0y + \lambda \\ y &= 1y + 0x \end{aligned}$$

Notice that the λ in the equation for x indicates that it is an accepting state. This gives us a set of simultaneous equations of the form $x = ax + b$ where the symbols a, b, x represent *languages* (rather than *numbers*). Here is a fact you can prove that tells how to solve such an equation.

Regular Equation Lemma: Let a and b be languages where $\lambda \notin a$. Then the equation $x = ax + b$ has exactly one solution, namely $x = a^*b$.

Proof: It is easy to check that $x = a^*b$ is a solution, since

$$ax + b = a(a^*b) + b = (aa^* + \lambda)b = a^*b = x$$

Assume that x is some solution; that is, x is a subset of Σ^* such that $x = ax + b$. We will show that x can only be a^*b .

Let $w \in a^*b$. Then $w = u_n u_{n-1} \cdots u_3 u_2 u_1 v$, where each u_i is in a and $v \in b$. Since $x = ax + b$, the union of ax and b , we know that $v \in x$. Since $ax \subseteq x$ and $u_1 \in a$, we have $u_1 v \in x$. Since $u_2 \in a$, we have $u_2 u_1 v \in x$. After n steps, we conclude that $u_n u_{n-1} \cdots u_3 u_2 u_1 v = w \in x$. Therefore $a^*b \subseteq x$.

It remains to show that $x \subseteq a^*b$. Suppose not, and let w be a string of minimal length that is in x but not in a^*b . Since $w \notin a^*b$, we know that $w \notin b$. Therefore, $w \in ax$. Let $w = uv$, where $\lambda \neq u \in a$ and $v \in x$. Since $|v| < |x|$, we conclude that $v \in a^*b$. But that means that $w = uv$ is also in a^*b , a contradiction.

Now suppose we are given a FSA M with n states, including start state s . We can use it to write down a set of n simultaneous equations telling about the strings that lead from each state to an accepting state. Let x be any state other than s . We use the lemma to express $x = a^*b$ in terms of the alphabet symbols of Σ and the other state variables. Substituting a^*b for each occurrence of x in the other equations gives us $n - 1$ equations with $n - 1$ variables. Repeating this process $n - 1$ times leaves us with one equation in one variable s . We then use the lemma to solve that equation and obtain a regular expression for the language $\mathcal{L}(M)$! This establishes an important theorem.

Theorem: $\text{FSA} \subseteq \text{REG}$; that is, the language recognized by a finite state acceptor can be represented by a regular expression.

Let us try applying this technique to Problem (2.4). From

$$s = 2s + 0u + 1v$$

we get

$$s = 2^*(0u + 1v)$$

From

$$w = \{1, 2\}w + 0s$$

we get

$$w = \{1, 2\}^*0s = \{1, 2\}^*02^*(0u + 1v)$$

Thus

$$u = \{1, 2\}u + 0w = \{1, 2\}^*0w = \{1, 2\}^*0\{1, 2\}^*02^*(0u + 1v)$$

Applying the lemma to solve for u again,

$$u = (\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1v$$

and therefore

$$\begin{aligned} s &= 2^*(0u + 1v) \\ &= 2^*(0(\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1v) + 1v \\ &= 2^*(0(\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1 + 1)v \end{aligned}$$

This expression has the form $s = av$, where a represents all strings that take M from state s to state v . It remains only to find an expression for v .

Solving $y = 1y + 0x$ gives $y = 1^*0x$, so we have

$$\begin{aligned} x &= 2x + 0y + \lambda = 2^*(0y + \lambda) \\ &= 2^*(01^*0x + \lambda) = 2^*01^*0x + 2^*\lambda \end{aligned}$$

so that

$$x = (2^*01^*0)^*2^*\lambda = (2^*01^*02^*)^*\lambda$$

The last equation gives us

$$v = 2x + 1y = 2x + 11^*0x = (2 + 11^*0)x = (2 + 11^*0)(2^*01^*02^*)^*\lambda$$

Notice that the first expression tells how to get from v to x , while the second tells how to return to x from x . Combining this with the expression for s gives the final solution:

$$\begin{aligned} s &= 2^*(0(\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1 + 1)v \\ &= 2^*(0(\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1 + 1)(2 + 11^*0)(2^*01^*02^*)^*\lambda \end{aligned}$$

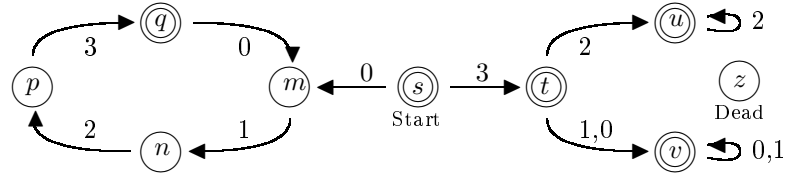
A regular expression for the language $\mathcal{L}(M)$ is therefore given by

$$2^*(0(\{1, 2\}^*0\{1, 2\}^*02^*0)^*\{1, 2\}^*0\{1, 2\}^*02^*1 \sqcup 1)(2 \sqcup 12^*0)(2^*01^*02^*)^*$$

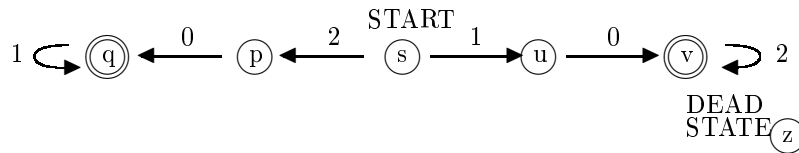
Problems

Find a regular expression for the language recognized by each FSA, by solving equations.

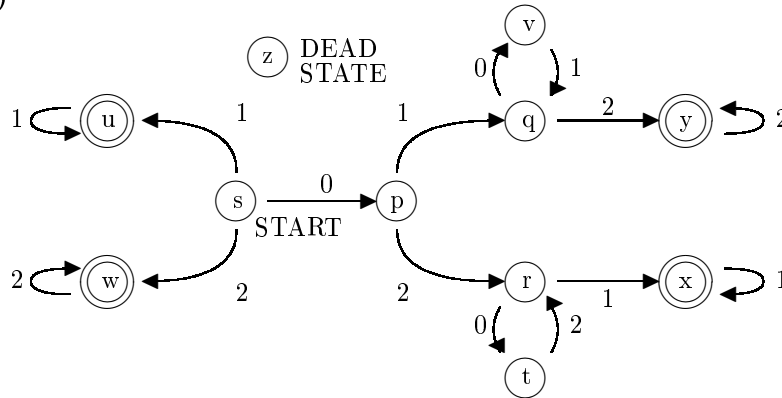
(4.1)



(4.2)



(4.3)



4.2 Regular Expressions Into Regular Grammars

Formal grammars are classified according to the complexity of their production rules. The simplest form of a production rule is a rule of the form

$$A \rightarrow aB, \quad A \rightarrow a, \quad \text{or } A \rightarrow \lambda$$

where A and B are nonterminal symbols and a is a terminal symbol. Grammars which only have production rules of these two forms are called *regular grammars*. The set of languages generated by regular grammars is denoted by

$$\mathbb{RG} = \{ L \mid L = \mathcal{L}(G) \text{ for some regular grammar } G. \}$$

When we construct regular grammars, it is often convenient to notice that productions of the form

$$A \rightarrow B \text{ (where } A \text{ and } B \text{ are non-terminal)}$$

may be included, as they can be replaced by regular productions $A \rightarrow aC$ for each production $B \rightarrow aC$, by $A \rightarrow a$ for each production $B \rightarrow a$, and by $A \rightarrow \lambda$ for each production $B \rightarrow \lambda$.

For example, let $L \subseteq \{a, b, c\}^*$ consist of all strings containing at least one b , that either begin with a and end with c , or begin with c and end with a . We show that $L \in \text{REG}$ by writing down a regular grammar for L :

$$\begin{array}{lll} S \rightarrow A, C & & \\ A \rightarrow aU & U \rightarrow aU, cU, bV & V \rightarrow aV, bV, cV, c \\ C \rightarrow cX & X \rightarrow aX, cX, bY & Y \rightarrow aY, bY, cY, a \end{array}$$

The reader should check that this grammar does indeed generate the language L .

Problems

Find a regular grammar that generates each of the following languages:

(4.4) $\mathcal{L}(\alpha)$ where $\alpha = (a^* \sqcup b^* \sqcup c^*)^* b (a^* b^* c^*)^*$. (See problem (3.6).)

(4.5) All strings in $\{0, 1\}^*$ with no more than three 1s.

(4.6) All strings in $\{0, 1\}^*$ where the number of 1s is divisible by 3.

(4.7) $\mathcal{L}(\alpha)$ where $\alpha = ((a \sqcup b)^* (cc^*))^* \sqcup ((a \sqcup c)^* (bb^*))^*$

The example and problems above give us evidence that regular languages (those defined by regular expressions) can always be generated by regular expressions. This is, in fact, true in general.

Theorem: $\text{REG} \subseteq \text{RG}$; that is, every regular expression defines a language that can be generated by a regular grammar.

Proof: The recursive definition of regular expressions tells us that this will be true provided that we can do two things:

1. Find a regular grammar to generate the languages $\emptyset$, $\{\lambda\}$, and $\{a\}$ for each $a \in \Sigma$.
2. Given regular grammars G_α and G_β for the languages defined by the regular expressions α and β , explain how to use them to construct regular grammars for the languages defined by $\alpha\beta$, α^* , and $\alpha \sqcup \beta$.

Part (1.) is easy: $S \rightarrow aS$, for any $a \in \Sigma$, is a grammar that generates $\emptyset$. The grammar $S \rightarrow \lambda$ generates $\{\lambda\}$, and the grammar $S \rightarrow a$ generates $\{a\}$.

To describe an algorithm for part (2.), let S_α and S_β be the start states of G_α and G_β , and fix these two grammars so that they have no non-terminal symbols in common. Now we produce three new grammars G .

$\alpha\beta$: We take S_α as the start state of G , and all production rules of G_α and G_β as production rules for G . We then replace each rule $A \rightarrow a$ in G_α with $A \rightarrow aS_\beta$, where a is either λ or is in Σ .

α^* : We take S_α again as the start state, and all production rules of G_α as production rules for G . For each rule $A \rightarrow a$ in G_α , we add $A \rightarrow aS_\alpha$, where a is either λ or is in Σ . We also include $S_\alpha \rightarrow \lambda$ in G .

$\alpha \sqcup \beta$: We take all production rules of both G_α and G_β as production rules for G . We then add a new start state S for G , together with the production $S \rightarrow S_\alpha$ and $S \rightarrow S_\beta$.

To illustrate this algorithm, consider the regular expression

$$\alpha = ((00)^*(1 \sqcup 2)^*) \sqcup (012)^*$$

We recursively construct a regular grammar for the language $\mathcal{L}(\alpha)$, keeping track of the start symbol for each smaller grammar.

	Regular Expression	Start	Production Rules
(1)	(00)	A	$\{A \rightarrow 0B; B \rightarrow 0\}$
(2)	(00)*	A	$(1) \cup \{B \rightarrow 0A; A \rightarrow \lambda\}$
(3)	(1 $\sqcup$ 2)	C	$\{C \rightarrow 1; C \rightarrow 2\}$
(4)	(1 $\sqcup$ 2)*	C	$(3) \cup \{C \rightarrow 1C; C \rightarrow 2C; C \rightarrow \lambda\}$
(5)	((00)*(1 $\sqcup$ 2)*)	A	$((2) \sim \{B \rightarrow 0; A \rightarrow \lambda\}) \cup \{B \rightarrow 0C; A \rightarrow C\} \cup (4)$
(6)	(012)	D	$\{D \rightarrow 0E; E \rightarrow 1F; F \rightarrow 2\}$
(7)	(012)*	D	$(6) \cup \{F \rightarrow 2D; D \rightarrow \lambda\}$
(8)	((00)*(1 $\sqcup$ 2)*) $\sqcup$ (012)*	G	$(5) \cup (7) \cup \{G \rightarrow A; G \rightarrow D\}$

Expanding (8) and eliminating obviously redundant productions, the final grammar for α has starting symbol G and productions

$$\begin{aligned} G &\rightarrow A, D; \\ A &\rightarrow 0B, C; \\ B &\rightarrow 0A; \\ C &\rightarrow 1C, 2C, \lambda; \\ D &\rightarrow 0E, \lambda; \\ E &\rightarrow 1F; \\ F &\rightarrow 2D. \end{aligned}$$

Together with our previous theorem, we have now established that

$$\text{FSA} \subseteq \text{REG} \subseteq \text{RG}$$

Problems

Use the recursive method above to find regular grammars for the languages defined by the regular expressions

$$(4.8) \quad ((aa)^*(b \sqcup c)^*)^* \sqcup (abc)^*$$

$$(4.9) \quad ((012)^* \sqcup (345)^*)4^*$$

4.3 Regular Grammars Into NDFSAs

Given that $\text{FSA} \subseteq \text{REG} \subseteq \text{RG}$, it is tempting to try to prove that $\text{RG} \subseteq \text{FSA}$, as this would tell us that finite state acceptors, regular expressions, and regular grammars, all specify exactly the same languages! It turns out that this is not as easy as it might seem from testing a simple example. Let G be the regular grammar with terminal alphabet $\Sigma = \{0, 1, 2\}$, non-terminal alphabet $N = \{S, A, B, C, D, E\}$, and production rules

$$\begin{aligned} S &\rightarrow 0S, 1A, 2B; \\ A &\rightarrow 1A, 0C; \\ B &\rightarrow 2B, 0C; \\ C &\rightarrow 1D; \\ D &\rightarrow 2E; \\ E &\rightarrow \lambda \end{aligned}$$

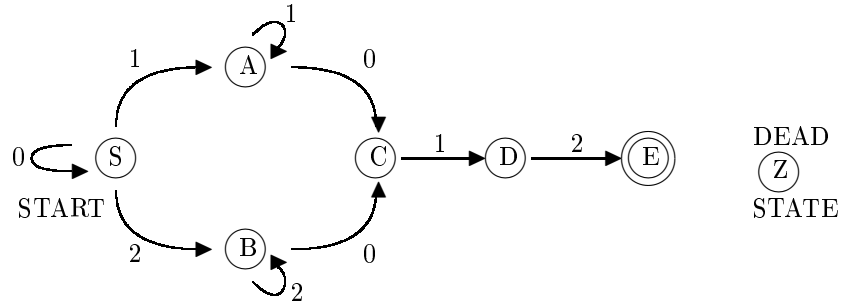
We will construct the finite state acceptor

$$M = \langle \{0, 1, 2\}, \{S, A, B, C, D, E\}, S, \{E\}, \delta \rangle$$

where we take

- the input alphabet $\{0, 1, 2\}$ to be the terminal alphabet of G ;
- the states $\{S, A, B, C, D, E\}$ to be the non-terminal symbols of G ;
- the initial state S to be the start symbol of G ;
- the accepting state E is the only state I for which $I \rightarrow \lambda$ in G ;
- the transition function $\delta(I, x) = J$ if $I \rightarrow xJ$ is in G .

This gives us the transition diagram below.



We ask the reader to check that $\mathcal{L}(G)$ and $\mathcal{L}(M)$ are both the language $0^*(11^* \cup 22^*)012$.

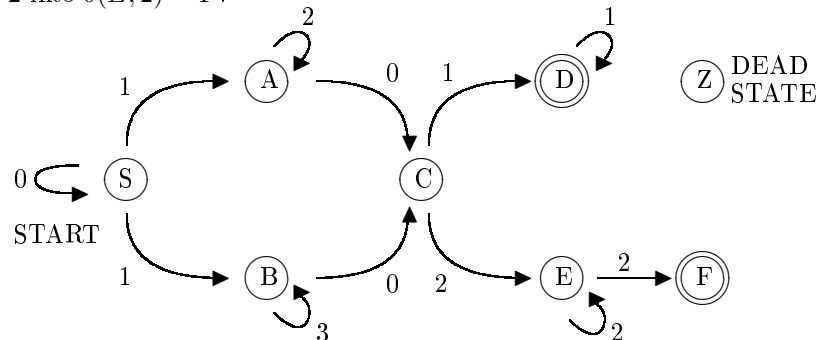
A second example raises some serious problems. Let G be the grammar with terminal alphabet $\{0, 1, 2\}$, non-terminal alphabet $\{S, A, B, C, D, E\}$, start symbol S , and productions

$$\begin{aligned}
 S &\rightarrow 0S, 1A, 1B; \\
 A &\rightarrow 2A, 0C; \\
 B &\rightarrow 3B, 0C; \\
 C &\rightarrow 1D, 2E; \\
 D &\rightarrow 1D, \lambda; \\
 E &\rightarrow 2E, 2
 \end{aligned}$$

You can check that this grammar generates the language

$$\mathcal{L}(G) = 0^*1(2^* \cup 3^*)0(11^* \cup 22^*)$$

If we follow the same construction for M , we face several dilemmas. For example, is $\delta(S, 1)$ supposed to be A or B ? (We have both $S \rightarrow 1A$ and $S \rightarrow 1B$.) A straightforward application of the above procedure gives us the peculiar diagram below. We have added accepting state F , and translated $E \rightarrow 2$ into $\delta(E, 2) = F$.



A string presented to this finite state “machine” can be executed in various different ways, as there are different choices when two arrows have the same

label. Notice, however, that the set of strings for which *some possible* sequence of choices leads to an accepting state is exactly $\mathcal{L}(G)$! This is an example of what is called a *non-deterministic finite state acceptor* (NDFSA). Formally, a NDFSA is a quintuple

$$N = \langle \Sigma, Q, s, Y, \delta \rangle$$

defined exactly as we defined a FSA, except that the transition function δ maps $Q \times \Sigma$ into the set of all *subsets* of Q ; that is, $\delta(q, a) \subseteq Q$ for $q \in Q$ and $a \in \Sigma$. In state q reading input symbol a , the action of N is not determined in advance; all we know is that it will go into some state in the set $\delta(q, a)$. The set $\delta(q, a)$ is illustrated in the transition diagram as the set of states pointed to by some arrow leaving q with label a . If N starts in state s reading a string $w \in \Sigma^*$, there might be many different possible runs of N , each with a different sequence of choices of transitions. We say that N *accepts* the string w if *some* possible run of N terminates in an accepting state of Y . As usual,

$$\mathcal{L}(N) = \{w \in \Sigma \mid N \text{ accepts } w\}$$

and the set of languages $\mathcal{L}(N)$ recognized by some NDFSA N is denoted by

$$\text{NDFSA} = \{\mathcal{L}(N) \mid N \text{ is a NDFSA}\}$$

The above NDFSA illustrates these ideas. Here, $\delta(S, 1) = \{A, B\}$ and $\delta(E, 2) = \{E, F\}$. Reading the string $w = 13022$, it could terminate in either E , F , or Z . Since one of these (F) is accepting, N accepts w and therefore $w \in \mathcal{L}(N)$. The language $\mathcal{L}(N)$ is easily checked to be exactly

$$\mathcal{L}(G) = 0^*1(2^* \cup 3^*)0(11^* \cup 222^*)$$

which is therefore in NDFSA. The method we used to construct this NDFSA from the grammar G illustrates our next theorem.

Theorem: $\text{RG} \subseteq \text{NDFSA}$; that is, for every regular grammar G , there is a non-deterministic finite state acceptor N' such that $\mathcal{L}(G) = \mathcal{L}(N')$.

Proof: Given the regular grammar $G = \langle \Sigma, N, S, \rightarrow \rangle$, let $N' = \langle \Sigma, Q, S, Y, \delta \rangle$ where

- the input alphabet Σ of N' is the terminal alphabet of G ;
- the set $Q = N \cup \{F, Z\}$ of states of N' consists of the non-terminal symbols of G together with a new (accepting) state F and a new dead state Z ;
- the initial state S of N' is the start symbol of G ;
- the set Y of accepting states of N' consists of F together with all states I for which $I \rightarrow \lambda$ is in G .
- the transition function δ of N' is defined by taking $\delta(I, x)$ to contain J if $I \rightarrow xJ$ is in G , to contain F if $I \rightarrow x$ is in G , and to contain I if $I \rightarrow \lambda$ is in G .

Then N' will be a NDFSA such that $\mathcal{L}(N') = \mathcal{L}(G)$.

We can summarize the central facts that we have discovered in a single inequality:

$$\boxed{\text{FSA} \subseteq \text{REG} \subseteq \text{RG} \subseteq \text{NDFSA}}$$

Problems

For each regular grammar G below, construct a NDFSA N such that $\mathcal{L}(N) = \mathcal{L}(G)$. Using the NDFSA, find a regular expression for the language $\mathcal{L}(G)$.

(4.10) $S \rightarrow 1A, 1B; \quad A \rightarrow 1A, 1D; \quad B \rightarrow 2B, 2; \quad D \rightarrow 3D, \lambda$

(4.11) $S \rightarrow 0A, 0B, 0C; \quad A \rightarrow 1A, 1B; \quad B \rightarrow 2B, 2C; \quad C \rightarrow 3C, 0, 1, 2$

(4.12)

$$\begin{array}{lll} S \rightarrow 0A, 0B, 0C; & & \\ A \rightarrow 1A, 1D, 1B; & B \rightarrow 2B, 2E, 2C; & C \rightarrow 3C, 3F, 3A; \\ D \rightarrow 0D, \lambda; & E \rightarrow 0E, \lambda; & F \rightarrow 0F, \lambda \end{array}$$

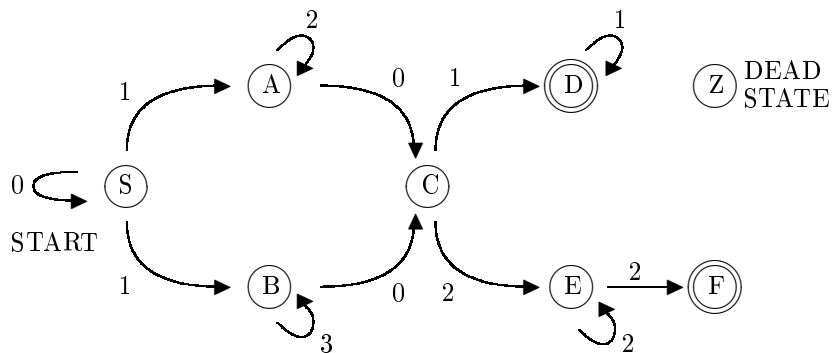
4.4 NDFSAs into (D)FSAs

A non-deterministic finite state acceptor N implicitly gives us a Test Algorithm to determine if a string w is in the language $\mathcal{L}(N)$. But the algorithm it gives is awkward and inefficient, as it requires that we do some kind of tree search to examine all possible runs of N , reading w to see if one ends in an accepting state. It would be nice to have a more direct algorithm to determine if N accepts an arbitrary string w .

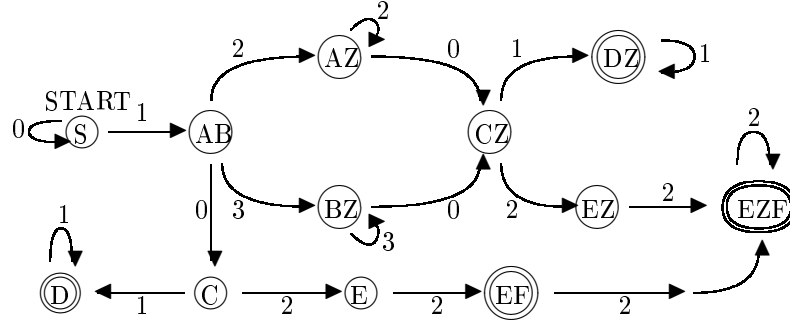
Consider the NDFSA N for the language

$$\mathcal{L}(N) = 0^*1(2^* \cup 3^*)0(11^* \cup 222^*)$$

that we constructed above from a grammar for this language:



In order to know if a string w is accepted by N , we must keep track of all states that N could possibly be in as it reads w . Starting in state S reading 0, the new state must be S . But reading a 1, the new state could be either A or B . In order to keep track of this information, we draw a different diagram.



Seeing 0 in starting state S , it can only go back to S . Seeing 1, it can go to either A or B . If it is in A or B and sees 2, it will go to either A or Z , and so forth. Reading the string $w = 001330222$, we see that N can end in any one of the states in the set $\{E, Z, F\}$. Since one of these, F , is an accepting state, we know that N accepts w . On the other hand, the diagram tells us that N reading $u = 0012202$ will end in a state in the set $\{E, Z\}$. Since this set does not contain an accepting state, N does not accept u .

Now if we look at this second diagram, we see that it is precisely the transition diagram for a (deterministic) finite state acceptor M for the language $\mathcal{L}(N)$! The states Q^M of M are the following *sets* of states of N :

$$\{ \{S\}, \{A, B\}, \{A, Z\}, \{B, Z\}, \{C, Z\}, \{D, Z\}, \{E, Z\}, \{E, Z, F\}, \{Z\} \}$$

The starting state is $S^M = \{S\}$, the transition function is given by

$$\delta^M(X, a) = \{ \delta(x, a) \mid x \in X, a \in \{0, 1, 2, 3\} \}$$

where $X \in Q^M$, and the accepting states are $\{D, Z\}$ and $\{E, Z, F\}$. This example illustrates an algorithm that can be applied to any NDFSA.

Theorem: $\text{NDFSA} \subseteq \text{FSA}$; that is, for every non-deterministic finite state acceptor N , there is a (deterministic) finite state acceptor M such that $\mathcal{L}(M) = \mathcal{L}(N)$.

Proof: Given $N = \langle \Sigma, Q, s, Y, \delta \rangle$, let $M = \langle \Sigma, Q^M, s^M, Y^M, \delta^M \rangle$ where

- $Q^M = \mathcal{P}(Q)$ is the set of all subsets of Q ;
- $s^M = \{s\}$;
- $Y^M = \{X \subseteq Q \mid X \cap Y \neq \emptyset\}$;
- $\delta^M(X, a) = \bigcup \{\delta(x, a) \mid x \in X\}$.

Then M , reading a string $w \in \Sigma^*$, will end in a state of Y^M if and only if some possible run of N will end in a state of Y . Thus M and N accept exactly the same strings.

Notice that in the example above, N has 8 states, so that using this construction, M will have $2^8 = 256$ states. In practice, it is not necessary to use all of the states of M , since most of them can never be reached from its start state. In the example, there are only 14 of the 256 states of M that are accessible from the start state.

This observation suggests that in general, NDFSAs are much more compact and easy to construct than the corresponding FSAs. On the other hand, they are much more cumbersome to use, since all the work of finding all possible paths through the diagram is done in advance when we construct the FSA.

Combining all of the theorems that we have proven, we conclude that the four classes of languages we have studied are in fact all the same:

$$\boxed{\text{FSA} = \text{REG} = \text{RG} = \text{NDFSAs}}$$

These languages are generally referred to as *regular languages*. Each of them has Generating Algorithm specifications by regular expressions and by regular grammars, and also has Test Algorithm specifications by a FSA and a NDFSAs.

From the fact that $\text{FSA} \subseteq \text{REG}$ and that $\text{REG} \subseteq \text{RG}$, it follows that $\text{FSA} \subseteq \text{RG}$. It is worth noticing that there is a very simple algorithm to construct a regular grammar G directly from a finite state acceptor M without constructing a regular expression along the way. We take the *states* of M (perhaps with simplified, one character names) as the *non-terminal symbols* of G .

We can then translate the transition diagram of M into production rules for G in exactly the same way that we translated them into the equations we needed to solve in order to produce a regular expression: If

$$\delta(x, a) = x, \quad \delta(x, b) = u, \quad \delta(x, c) = v$$

in M , for example, we can write down productions

$$x \rightarrow ax; \quad x \rightarrow bu; \quad x \rightarrow cv$$

for G instead of writing the equation $x = ax + bu + cv$. (Writing down the grammar is like writing down the equations for a regular expression without solving them!)

Problems

(4.13), (4.14), (4.15) For each of the NDFSAs that you constructed in problems (4.10), (4.11), and (4.12), use the above algorithm to construct a (D)FSA which recognizes the same language.

(4.16), (4.17), (4.18) For each of the FSAs that you constructed in problems (4.13), (4.14), and (4.15), construct a regular grammar which generates the

language recognized by the FSA. In order to do this, it will be convenient to redraw each FSA with a new single letter name for each state. These single letters will then become the nonterminal symbols of your grammar. If all your steps are correct, the resulting grammars you get will be equivalent to the grammars you started with in problems (4.10), (4.11), and (4.12).

5 Machine Minimization

When we write a computer program, our first task is to put together a version which runs properly and which we believe to be correct. Once this has been achieved, it is often important to revise the program in order to make it as clean, simple, and understandable as possible. Doing so makes it much easier to be confident of its correctness, and makes it much less difficult to maintain and modify it in the future. But the general problem of reducing a program to an equivalent program of minimal complexity is in general quite difficult, and algorithms to do so are extremely rare. In general, it is not even easy to find a practical way to measure the complexity of a program.

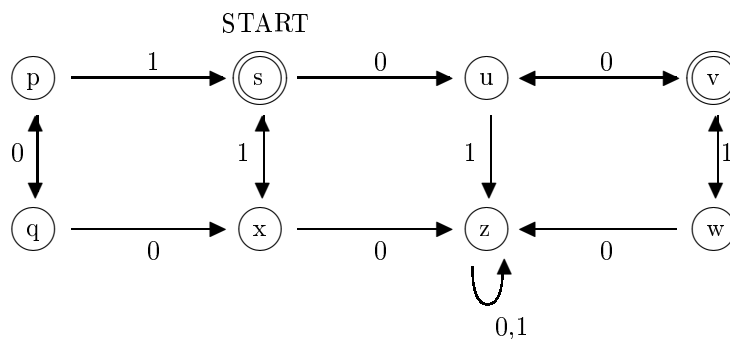
Once again, the finite state acceptor model of a simple computing machine gives us a context in which these problems can be fully resolved, providing a prototype of what we would optimally seek for more complex machines. In this chapter, we will see how to construct a minimal finite state acceptor for any regular language. As a by-product of this investigation, we will find a simple method to show that many more complex languages are *not* regular.

5.1 Minimal Finite State Acceptors

The *number of internal states*, $|Q|$, of a finite state acceptor $M = \langle \Sigma, Q, s, \delta, Y \rangle$ provides us with a useful measure of the *complexity* of M . Our problem is to find a method to generate from a regular language L a finite state acceptor M such that

- M recognizes L ; and
- if M' is any FSA which recognizes L , then M' has at least as many states as M .

We can then view M as a minimal FSA that recognizes L . For example, let L_0 be the regular language recognized by the FSA below.



It is easy to check that $L_0 = (\{00\} \cup \{11\})^*$.

Problems

- (5.1) Construct an FSA M_0 that also recognizes L_0 , but has only 4 states.
- (5.2) Let M'_0 be any finite state acceptor that recognizes L_0 , and let s' be its start state. Looking at which states are accepting and which are not, explain why the states s' , $\delta(s', 0)$, $\delta(s', 1)$, and $\delta^*(s', 10) = \delta(\delta(s', 1), 0)$ must all be distinct. Thus M'_0 has at least as many states as M_0 .

Our goal is to find a general procedure that will produce, from any regular language L , a minimal FSA M that recognizes L .

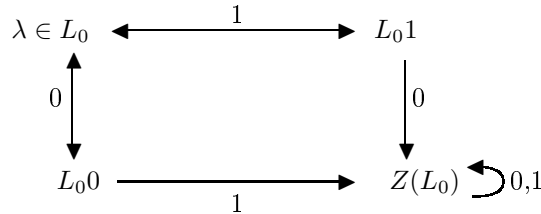
To begin with, let $L \subseteq \Sigma^*$ be an arbitrary language. We define a special equivalence relation $\equiv_L$ on the set Σ^* based on the language L . If $w \in \Sigma^*$, let $R_w = \{z \in \Sigma^* \mid wz \in L\}$. We call this the *right-set* for w . Notice that if w does not happen to be an initial segment of a string in L , then $R_w = \emptyset$. We take $Z(L) = \{w \in \Sigma^* \mid R_w = \emptyset\}$ to be the set of strings which are not an initial segment of any member of L . We use these ideas to define an equivalence relation of Σ^* : for $x, y \in \Sigma^*$,

$$x \equiv_L y \text{ if } R_x = R_y$$

In the above example of $L_0 = (\{00\} \cup \{11\})^*$, we see that $R_0 = R_{110} = R_{00110}$ consists exactly of the set $0L_0 = \{0u \mid u \in L_0\}$. Thus the equivalence class $[0]$ of 0 under $\equiv_{L_0}$ is exactly L_00 . On the other hand, $01 \in Z(L_0)$ so $R_{01} = \emptyset$. Continuing in this way, we find that for each $w \in \{0, 1\}^*$, the right-set R_w is either L_0 , $0L_0$, $1L_0$, or $\emptyset$. These choices give us exactly four different $\equiv_{L_0}$ -classes:

$$\begin{aligned} [\lambda] &= L_0 \\ [0] &= L_00 \\ [1] &= L_01 \\ [01] &= \Sigma^* \sim (L_0 \cup L_00 \cup L_01) = Z(L_0) \end{aligned}$$

These four $\equiv_{L_0}$ -classes have a curious property. If any member of L_00 is extended on the right by adding a 0, the result is always a string in L_0 . If any member of L_00 is extended on the right with a 1 instead, we get a string in $Z(L_0)$. These facts are summarized in the following diagram.



Since $\lambda \in L_0$, we can take any string $w \in \{0, 1\}^*$ and find which class w is in by tracing the diagram, building w by starting with λ and successively adding 0s or 1s on the right until we have w . Our ending point will be the $\equiv_{L_0}$ -class of w . If we land in L_0 , then $w \in L_0$; otherwise, it is not. What we have done is to build a four-state FSA, whose start state and only accepting state is L_0 , that recognizes L_0 !

Can we repeat this construction with any language L , using the $\equiv_L$ -classes to find an FSA for L ? There is one important hitch, as the following example shows.

(5.3) Let $L_+ = \{a^n b^n \mid n = 1, 2, 3, \dots\}$. Show that there are infinitely many $\equiv_{L_+}$ -classes by making an infinite list of strings, no two of which are equivalent by $\equiv_{L_+}$.

Obviously, we cannot use the $\equiv_L$ -classes as states of a *finite* state acceptor if there are infinitely many of them! But if the set of $\equiv_L$ -classes is finite, then this construction will always work.

Standard FSA Theorem: Suppose that $L \subseteq \Sigma^*$ is a language such that Σ^* has only finitely many different $\equiv_L$ -classes. Then L is recognized by an FSA which has exactly as many states as there are $\equiv_L$ -classes in Σ^* .

Proof: Let $M = \langle \Sigma, Q, s, Y, \delta \rangle$, where

- Q is the (finite) set of all $\equiv_L$ -classes;
- $s = [\lambda]$ is the start state;
- $Y = \{[x] \mid x \in L\}$ (which is unambiguously defined since you can check that for all $x, y \in \Sigma^*$, if $x \in L$ and $x \equiv_L y$, then $y \in L$);
- $\delta([x], a) = [xa]$ (which is unambiguously defined since you can check that for all $x, y \in \Sigma^*$ and $a \in \Sigma$, if $x \equiv_L y$, then $xa \equiv_L ya$).

This defines an FSA M with exactly as many states as there are $\equiv_L$ -classes (since its states *are* the $\equiv_L$ -classes!). To see which strings w are accepted by M , we need to know when $\delta^*(s, w) \in Y$. If $w = a_1 a_2 \dots a_n \in \Sigma^*$, then

$$\begin{aligned} \delta^*(s, w) &= \delta^*([\lambda], a_1 a_2 \dots a_n) = \delta^*(\delta([\lambda], a_1), a_2 \dots a_n) \\ &= \delta^*([a_1], a_2 \dots a_n) = \delta^*([a_1 a_2], \dots a_n) = \dots = [a_1 a_2 \dots a_n] = [w] \end{aligned}$$

Thus $\delta^*(s, w) \in Y$ if and only if $w \in L$, so M recognizes L .

The FSA M constructed in this proof is called the *standard* FSA for L , as it is constructed in a uniform way from the language L itself.

In case L is a regular language, the relation $\equiv_L$ is very special. To see this, let $M = \langle \Sigma, Q, s, \delta, Y \rangle$ be an arbitrary FSA which recognizes L . We now define a second equivalence relation on Σ^* , this one *based on the machine* M . For $x, y \in \Sigma^*$,

$$x \equiv_M y \text{ if } \delta^*(s, x) = \delta^*(s, y)$$

Thus two strings are equivalent if M lands in the same state reading one as reading the other. A state q of an FSA M is said to be *accessible* if there is *some* string x such that $\delta^*(s, x) = q$; that is, M can reach q from the start state. For each accessible state q , we have a distinct $\equiv_M$ -class consisting of all strings x such that $\delta^*(s, x) = q$.

Myhill-Nerode Theorem: Let L be a regular language. Then the set of $\equiv_L$ -classes is finite, and every FSA that recognizes L has at least as many states as the standard FSA for L has.

Proof: Let M be an FSA that recognizes L , and let $q_1, q_2, \dots, q_n$ be a list of the accessible states of M . Then $K_{q_1}, K_{q_2}, \dots, K_{q_n}$ is a list of all of the $\equiv_M$ -classes. Since M has at least n states, it will suffice to show that the number of $\equiv_L$ -classes in Σ is no more than n . We will do this by showing that each $\equiv_L$ -class is a union of $\equiv_M$ -classes. (This fact is sometimes stated by saying that the partition induced by $\equiv_M$ is a *refinement* of the partition induced by $\equiv_L$.)

Let $x \equiv_M y$. We must show that $x \equiv_L y$. Since $x \equiv_M y$, we have for all $z \in \Sigma^*$, that $\delta^*(s, x) = \delta^*(s, y)$, and consequently

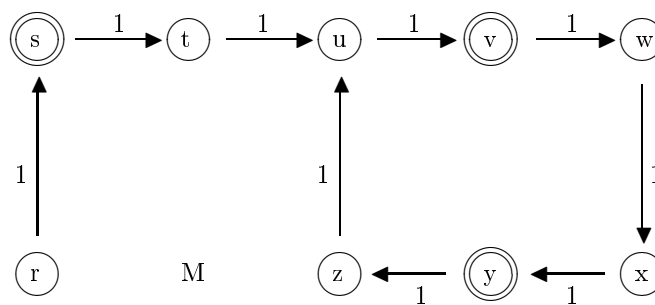
$$\delta^*(s, xz) = \delta^*(\delta^*(s, x), z) = \delta^*(\delta^*(s, y), z) = \delta^*(s, yz)$$

Thus $\delta^*(s, xz) \in Y$ if and only if $\delta^*(s, yz) \in Y$, and therefore $xz \in L$ if and only if $yz \in L$. But this tells us that $x \equiv_L y$.

Our original finite state acceptor M for L_0 , for example, has six accessible states s, u, v, w, z , and x , which give us a partition of each of the $\equiv_{L_0}$ -classes into $\equiv_M$ -classes: $L_0 = K_s \cup K_v$, $L_01 = K_u \cup K_w$, $L_00 = K_u$, and $Z(L_0) = K_z$.

Problems

For each of the following regular languages, look at the different right-sets to determine the distinct $\equiv_L$ -classes. Then construct the standard FSA for L .



(5.4) $L = 0^*1^*2^*$

$$(5.5) \quad L = 00(1^* \cup 2^*)$$

$$(5.6) \quad L = (1^* \cup 2^*)00$$

$$(5.7) \quad L = 111 \cup (020)^*$$

$$(5.8) \quad L = (111)^*1$$

(5.9) Above is an FSA M for the language $L = (111)^*1$. Find a regular expression for each $\equiv_M$ -class: K_r , K_s , K_t , K_u , K_v , K_w , K_x , K_y , and K_z .

(5.10) Express each $\equiv_L$ -classes as a union of $\equiv_M$ -classes, thereby illustrating the proof that M must have at least as many states as there are $\equiv_L$ -classes.

5.2 Languages That Are Not Regular

Combining the two previous theorems gives us a very useful characterization of regular languages which will allow us to say which languages are *not* regular.

Myhill-Nerode Corollary: A language $L \subseteq \Sigma^*$ is regular if and only if the set of $\equiv_L$ -classes in Σ^* is finite.

Proof: If L is regular, then the M-N Theorem tells us that the set of $\equiv_L$ -classes is finite. If the set of $\equiv_L$ -classes is finite, then the Standard FSA Theorem shows us how to construct the standard FSA for L which guarantees that L is a regular language.

For example, this corollary tells us for certain that the language $L_ =$ in problem (5.3) is *not* a regular language, because there are infinitely many $\equiv_{L_ =}$ -classes. Using the same ideas, we can formulate a direct explanation as to why this and certain other languages are not regular, without reference to equivalence relations.

Example 1: The language $L_ = = \{a^n b^n \mid n = 1, 2, 3, \dots\}$ is not regular.

Let M be any FSA. We will show that M does not recognize $L_ =$ by finding two strings, $u \in L_ =$ and $v \notin L_ =$, such that M ends in the same state reading either u or v . Imagine that we ask M to read strings of as: $a, a^2, a^3, a^4, \dots$, starting in its start state s . Let $q_n = \delta^*(s, a^n)$ be its ending state when it reads a^n . Since M has only finitely many states, there must be two different indices $m < n$ such that $q_m = q_n$. Taking $u = a^m b^m$ and $v = a^n b^m$, we have

$$\delta^*(s, u) = \delta^*(s, a^m b^m) = \delta^*(q_m, b^m) = \delta^*(q_n, b^m) = \delta^*(s, a^n b^m) = \delta^*(s, v)$$

This tells us that either M accepts both $u \in L_ =$ and $v \notin L_ =$, or it accepts neither. In either case, M is not an FSA for $L_ =$.

Example 2: The language L_{alg} of all meaningful algebraic expressions over $\Sigma = \{w, x, y, z, (,), +, -, \cdot, \div\}$ is not regular.

Again, let M be any FSA. Notice that in a string $w \in L_{alg}$, the number of “(”s and “)”s must be exactly the same. This time, imagine M beginning in its start state s reading $(, (, ((, (((, (((, \dots$. Again, there must be $m < n$ such that M is in the same state after reading m “(”s as after reading n “(”s. Let $u = ((\dots((x+x)+x)\dots+x)+x) \in L_{alg}$ where there are n “(”s before the first “ $x+$ ”, followed by n “ $+x$ ”s. Now let v be obtained from u by removing the first $n-m$ “(”s, leaving only m “(”s. Then M , starting in s , will end in the same state reading u or reading v , so it either accepts both or rejects both. Thus M does not recognize L_{alg} .

Problems

Use an argument like the ones given above to show that each of the following languages is not regular.

(5.11) $L = \{0^n 10^n \mid n = 1, 2, 3, \dots\}$

(5.12) $L = \{w c w \mid w \in \{a, b\}^*\}$

(5.13) L_{re} is the set of regular expressions over the alphabet $\{a, b, c\}$.

(5.14) $L \subseteq \{0, 1\}$ is the set of w containing the same number of a s as b s.

6 PDAs and CFGs

As we have seen, the beauty of regular languages is that they can be so thoroughly understood from so many different points of view. But we have also seen that many relatively simple languages are not regular. In this chapter, we will examine a larger class of languages which is broad enough to include, for example, many different computer languages, while still restrictive enough to be very well-understood. These are called *context-free languages*. They correspond to a model of computing machine called a *pushdown acceptor*, a machine which is significantly more powerful than a finite state acceptor, but is still simple enough to be very well-behaved.

6.1 Context-Free Languages

Regular grammars are very limited because they can only generate a string from left to right, one character at a time. As we have seen, for example, the language $L_ =$ is not regular because a regular grammar cannot produce a string of *as* followed by a string of *bs* and guarantee that there will be the same number of *as* as *bs*. In this section, we will study a slightly broader class of grammars that will allow us to generate many new languages, including most modern computer languages, but is restrictive enough to ensure that the constructed languages are still relatively simple.

We will say that a formal grammar $G = \langle \Sigma, N, S, \rightarrow \rangle$ is *context-free* if each production rule of G has the form $A \rightarrow v$ where $A \in N$ and $v \in (\Sigma \cup N)^*$. If G is context-free, we say that the language $\mathcal{L}(G)$ is a *context-free language*, and we denote the set of all context-free languages by $\mathbf{CFL}$.

Regular grammars, for example, are all context-free. In contrast, look at the example of a grammar for email addresses. There you see production rules like

$$LOC.edu \rightarrow newpaltz.edu$$

and

$$COMP.newpaltz \rightarrow matrix.newpaltz$$

telling us that *LOC* can be replaced by *newpaltz* provided that it occurs in the context *LOC.edu*, and *COMP* can be replaced by *matrix* provided that it occurs in the context *COMP.newpaltz*. If such *context-sensitive* rules are allowed in a grammar, it turns out that much more complex languages can be generated.

Example 1: A very simple context-free grammar will generate the language $L_ = = \{a^n b^n \mid n = 1, 2, \dots\}$:

$$S \rightarrow aSb, \lambda$$

Example 2: The language L_{re} of all regular expressions over an alphabet Σ is generated by a context-free grammar which simply restates the definition

of “regular expressions”:

$$\begin{aligned} S &\rightarrow \lambda, \emptyset, a \text{ for each } a \in \Sigma \\ S &\rightarrow (S \sqcup S), (SS), S^* \end{aligned}$$

Example 3: As a more interesting example, let $L \subseteq \{a, b\}^*$ be the set of strings with the same number of a s as b s. Consider the context-free grammar G with production rules

$$S \rightarrow SaSbS, SbSaS, \lambda$$

Since G generates a s and b s in pairs, it clearly generates only strings in L . The question is whether or not *all* strings in L can be generated by G . We show that they can be, in two steps.

Step 1: We first show that if $\lambda \neq w \in L$, then there are $x, y, z \in L$ such that w is either $xaybz$ or $xb Yaz$. Consider the first and last letters in w . If $w = ayb$ or $w = bya$, then $y \in L$, and we take $x = \lambda = z$. Suppose w begins and ends with the same letter, say $w = aua$. Read w from left to right, keeping a count of the number of a s minus the number of b s. The count will look like this: $0, 1, \dots, -1, 0$. Since it must increase or decrease by one at each step, it must go from 1 to 0 at some point in between. Thus $w = aybva$, where the count is 0 after the b . Thus $ayb \in L$, so $y \in L$ and $va \in L$. We take $x = \lambda$ and $z = va$. This gives us $w = xaybz$, where $x, y, z \in L$.

Step 2: If $w \in L$ is λ , then G generates w . Otherwise, use Step 1 to write w , say, as $xb Yaz$. Now do the same with each of x, y , and z ; say we get $x = x_1ay_1bz_1$, $y = \lambda$, and $z = x_2by_2az_2$. Continuing in this way, we replace each successive x_i, y_i , and z_i by smaller and smaller strings in L until they are eventually replaced by λ . This gives us a way to produce w from G :

$$S \rightarrow SbSaS \rightarrow [SaSbS]b\lambda a[SbSaS] \rightarrow \dots \rightarrow xbyaz = w$$

(where “[” and “]” are not symbols in the string).

Problems

Write production rules for a context-free grammar for each of the following languages.

(6.1) $L = \{0^n 10^n \mid n = 1, 2, 3, \dots\}$

(6.2) $L = \{w^R cw \mid w \in \{a, b\}^*\}$, where w^R is the reverse of w .

(6.3) $L = \{ab^n c^n ab^m c^m a \mid n, m = 1, 2, 3, \dots\}$

(6.4) $L_{alg} \subseteq \{w, x, y, z, (,), +, -, \cdot, \div\}^*$ is the set of all meaningful algebraic expressions.

(6.5) $L_{w^R w} = \{w^R w \mid w \in \{a, b\}^*\}$

6.2 Pushdown Acceptors

The limitations of a finite state acceptor stem from the fact that it has no extended (long-term) memory. The states of the FSA act as a short-term memory, dictating different behavior in each different state. But it cannot, for example, recognize the language $L_=$, because it cannot remember how many *as* it has seen in order to compare the length of a string of *as* with that of a string of *bs*. We would like to extend the FSA model only far enough to allow some kind of extended memory.

What, then, is the simplest form of extended memory? Extended memory consists of a database which is organized in such a way that we are able to add data to it and retrieve data from it. The more sophisticated our management of the database, the more powerful the resulting computing facility. So what we are looking for is the *least*-sophisticated method we can think of for management of an arbitrarily large database. You can see such a system in action if you come to visit my office, and notice how I manage my mail. There you will see a stack of mail on my desk. When mail comes in, I put it on top of the stack. Perhaps I read some of it, and then dispense with it as necessary. The rest sits on top of the stack. During slack periods, when they arise, I work my way down the stack, dealing with the most recent mail first. During more busy periods, like the beginning, middle, and end of the term, the stack tends to get higher. Between terms, I always work my way to the bottom of the stack and clear my desk. Some people manage their refrigerators in the same way, but this is a practice I wouldn't recommend.

Now imagine a machine P , similar to a finite state acceptor, which is further endowed with a *pushdown stack*. The items on the stack are symbols from a pushdown alphabet Γ . We designate an input alphabet Σ , a set Q of internal states, and a start state s . We start P in state s , looking at the left-most symbol of an input string $w \in \Sigma^*$. The action of P will be determined by

1. the current internal state $q \in Q$;
2. the symbol from Γ at the top of the pushdown stack; and
3. the currently-read input symbol $a \in \Sigma$.

A transition function will tell P how to use 1., 2., and 3. to determine

1. what new state to enter; and
2. whether it should *push* $\downarrow$ a particular new symbol of Γ on top of the stack, or *pop* $\uparrow$ the top symbol off of the stack.

The machine P will halt if its transition function has no instruction for the current values of 1., 2., and 3. If P , reading $w \in \Sigma^*$, eventually halts with an empty pushdown stack having finished reading w , we say that P *accepts* w .

These ideas can be completely formalized. A *pushdown acceptor* (*PDA*) is a quintuple

$$P = \langle \Sigma, \Gamma, Q, s, \delta \rangle$$

consisting of an *input alphabet* Σ , a *pushdown alphabet* Γ , a set Q of *internal states*, an *initial state* $s \in Q$, and *transition function* δ , where δ is a function from a *subset* of $Q \times (\Gamma \cup \{\lambda, -\}) \times (\Sigma \cup \{\lambda, -\})$ into $Q \times ((\Gamma \times \{\downarrow, \uparrow\}) \cup \{-\})$. An input or stack reading of λ indicates that there are no further symbols; $-$ in the second position indicates that P is to ignore the pushdown stack; and $-$ in the third position indicates that P is not to read and advance the input tape.

For example, the instruction $\delta(q, g, a) = (r, g \uparrow)$ means that in state $q \in Q$ with $g \in \Gamma$ at the top of the stack reading input a , it will enter state r and remove g from the top of the stack. We will write this as

$$(q, g, a) \rightarrow (r, g \uparrow)$$

The instruction $\delta(q, -, a) = (r, g \downarrow)$ means that regardless of what is on the stack, if P is in state q reading a , then it will enter state r and add g to the top of the stack. We will, for compositional convenience, write the contents of the stack as a string $w \in \Gamma^*$ with *the top at the right and the bottom at the left*.

As usual, we will denote the *language recognized by* P as

$$\mathcal{L}(P) = \{w \in \Sigma^* \mid P \text{ accepts } w\}$$

and the set of languages recognized by pushdown acceptors as

$$\text{PDA} = \{\mathcal{L}(P) \mid P \text{ is a PDA}\}$$

Example 4: The long evasive language $L_{=} = \{a^n b^n \mid n = 1, 2, \dots\}$ is recognized by a very simple pushdown acceptor $P = \langle \{a, b\}, \{a\}, \{s, t\}, s, \delta \rangle$, where δ is given by

$$(s, -, a) \rightarrow (s, a \downarrow) \quad (s, a, b) \rightarrow (t, a \uparrow) \quad (t, a, b) \rightarrow (t, a \uparrow)$$

The action of P is seen by tracing its transitions, as we do here for two sample input strings, $u = aaabbb \in L_{=}$ and $v = aababb \notin L_{=}$.

State	Stack	Input	State	Stack	Input
s	λ	$aaabbb$	s	λ	$aababb$
s	a	$aabbb$	s	a	$ababb$
s	aa	$abbb$	s	aa	$babb$
s	aaa	bbb	t	a	abb
t	aa	bb		$HALT!$	
t	a	b			
t	λ	λ			

Here we see that P will accept u but not v .

Example 5: Let $L \subseteq \{a, b\}^*$ consist of all strings with the same number of a s as b s. We give here production rules for a PDA P that recognizes L :

$$\begin{array}{lll} (s, -, a) \rightarrow (s, a \downarrow) & (s, a, b) \rightarrow (s, a \uparrow) & (s, \lambda, b) \rightarrow (t, b \downarrow) \\ (t, -, b) \rightarrow (t, b \downarrow) & (t, b, a) \rightarrow (t, b \uparrow) & (t, \lambda, a) \rightarrow (s, a \downarrow) \end{array}$$

In state s , it will push as onto the stack and pop them off as it sees bs . In state t , it will push bs on the stack and pop them off as it sees as . It needs to have both options, depending upon whether or not the number of as read exceeds the number of bs read.

Problems

(6.6) Trace the action of the PDA in Example 5 above for the string $u = aabbbbbaaaaabbb \in L$ and $v = bbabaabb \notin L$.

Construct a pushdown acceptor for each of the following languages. In each case, trace the action of your PDA for the given string $u \in L$ and $v \notin L$.

(6.7) $L = \{0^n 10^n \mid n = 1, 2, 3, \dots\}$. Let $u = 000010000$ and $v = 0001001001$.

(6.8) $L = \{w^R cw \mid w \in \{a, b\}^*\}$ where w^R is the reverse of w . Let $u = baabacabaab$ and $v = aaabacabbaa$.

(6.9) $L = \{ab^n c^n ab^m c^m a \mid n, m = 1, 2, 3, \dots\}$ Let $u = abbccabbbccca$ and $v = abbcabbbccca$.

6.3 Epilogue

We have now gathered a bundle of context-free languages which are recognizable by a pushdown acceptor, but which are known not to be regular. If this is a representative sample, we would be tempted to guess that

$$\text{REG} \subsetneq \text{PDA} = \text{CFL}$$

Checking again, we have seen a couple of languages generated by a simple context-sensitive grammar for which we don't yet have a PDA: $L_{alg} \subseteq \{w, x, y, z, (,), +, -, \cdot, \div\}^*$ and $L_{w^R w} = \{w^R w \mid w \in \{a, b\}^*\}$. Both of these languages are somewhat problematic, but can be successfully handled with an adequate enhancement of the PDA model.

Example 6: It is convenient to first look at a slightly simplified version of L_{alg} .

Let L be the language given by the context-free grammar $S \rightarrow [SS], x$. This is the language of all proper algebraic expressions in a single variable x , with a single binary operation denoted by juxtaposition. A typical string in L might look like

$$w = [[[xx]x][x[xx]]][x[x[xx]]x]$$

A machine checking w for membership in L must, among other things, check that each left bracket matches a right bracket. Without too much difficulty, we can construct a "PDA" P to recognize L if we enhance the model by allowing P to read, push, and pop entire strings on the pushdown

stack at once, instead of just individual symbols. Here are the transitions for such a PDA which has a single state s .

$$\begin{aligned} (s, -, \downarrow) &\rightarrow (s, \downarrow) & (s, [, x) &\rightarrow (s, x \downarrow) \\ (s, x, x) &\rightarrow (s, x \uparrow) & (s, [, \downarrow) &\rightarrow (s, \downarrow \uparrow) & (s, [x, \downarrow) &\rightarrow (s, [x \uparrow) \end{aligned}$$

The last instruction utilizes our enhancement. If P finds $[x$ at the top of the stack (meaning the x is the top symbol and $[$ is just beneath it) and reads input $]$, then it will pop the *string* $[x$ off the stack. To illustrate the actions of P , we trace them for the input string $w = [[[[[xx]x][xx]]x]$:

State	Stack	Input	State	Stack	Input
s	λ	$[[[[[xx]x][xx]]x]$	s	$[[[x$	$][xx]]x]$
s	$[$	$[[[xx]x][xx]]x]$	s	$[[$	$[xx]]x]$
s	$[[$	$[[xx]x][xx]]x]$	s	$[[[$	$xx]]x]$
s	$[[[$	$[xx]x][xx]]x]$	s	$[[[x$	$x]]x]$
s	$[[[[$	$xx]x][xx]]x]$	s	$[[[[$	$]]x]$
s	$[[[[[x$	$x]x][xx]]x]$	s	$[[[[$	$]x]$
s	$[[[[[$	$]x][xx]]x]$	s	$[[[[$	$x]$
s	$[[[[[$	$x][xx]]x]$	s	$[x$	$]$
			s	λ	λ

This shows that w , but no initial segment of w , is in L .

In general, we can enhance the notion of a PDA by allowing P to search for an arbitrary string $u \in \Gamma^*$ at the top of its stack, and replace it by another arbitrary string $v \in \Gamma^*$. We would write this, for example, as $(q, u, a) \rightarrow (r, v)$, meaning that P , in state q with u at the top of the stack reading input a , should go into state r and replace u with v . It turns out that it is not difficult to show that this kind of enhancement can actually be achieved by an ordinary PDA, simply by adding a sequence of new states which cause P to pop u , character by character, and then push v , character by character.

Example 7: Now, consider $L_{w^R w}$. We might imagine a PDA checking for membership in $L_{w^R w}$ by pushing each symbol of w^R onto the stack, and then popping them back off the stack provided they matched the incoming symbol from w . There is one problem: without the center marker c , how do we know when to stop pushing and start popping?! This question can be circumvented by using a *non-deterministic acceptor* P :

$$\begin{aligned} (s, -, a) &\rightarrow (s, a \downarrow) & (s, -, a) &\rightarrow (t, a \downarrow) & (t, a, a) &\rightarrow (t, a \uparrow) \\ (s, -, b) &\rightarrow (s, b \downarrow) & (s, -, b) &\rightarrow (t, b \downarrow) & (t, b, b) &\rightarrow (t, b \uparrow) \end{aligned}$$

In state s , the machine P pushes as and bs onto the stack as it reads them, and it can choose either to continue in state s or to move into state t . In state t , it pops symbols off the stack as long as they match the input

symbol. Like the non-deterministic FSA, we say that P *accepts* a string if there is some possible run of P that stops having read the entire input string and having emptied its stack.

Combining these two enhancements gives the notion of a *non-deterministic pushdown acceptor (NDPDA)*

$$P = \langle \Sigma, \Gamma, Q, s, \delta \rangle$$

which differs from a PDA in that its *transition function* δ is a function from a *subset* of $Q \times (\Gamma^* \cup \{\lambda, -\}) \times (\Sigma \cup \{\lambda, -\})$ into the collection of *subsets* of $Q \times (\Gamma^* \cup \{-\})$. The subset $\delta(q, g, a)$ is the set of all actions the P may take in state $q \in Q$ reading $a \in \Sigma \cup \{\lambda\}$ with $g \in \Gamma^*$ at the top of the stack. This model is now broad enough to capture all context-free languages.

Theorem: $\text{CFL} \subseteq \text{NDPDA}$; that is, every context-free language is recognized by some non-deterministic pushdown acceptor.

Proof: Let $G = \langle \Sigma, N, S, \rightarrow \rangle$ be a context-free grammar for a language L . We construct a NDPDA $M = \langle \Sigma, \Sigma \cup N, \{s, q\}, s, \delta \rangle$ with the transitions

$$(s, \lambda, -) \rightarrow (q, S) \quad (q, A, -) \rightarrow (q, u^R) \quad (q, a, a) \rightarrow (q, \lambda)$$

for each production rule $A \rightarrow u$ of G , and each $a \in \Sigma$. State s is only used to put the start symbol S on the stack and then enter state q , where P will remain. In state q , it will non-deterministically apply all possible production sequences from G , disposing of terminal symbols as they appear at the top of the stack. Notice that u must be put on the stack upside-down, so that its leftmost symbol will be at the top of the stack. If P is given $w \in \Sigma^*$, it will be able to finish reading w and empty its stack if and only if G is able to produce w from S .

This brings us to our final question: Is $\text{PDA} = \text{NDPDA}$? That is, is it always possible – as it is with finite state acceptors – to simulate a non-deterministic pushdown acceptor with a (deterministic) pushdown acceptor? To appreciate this problem, it is worth trying to see if you can find a PDA for the language $L_{w^R w}$. In fact, there is no such PDA: *non-deterministic push-down acceptors are not, in general, equivalent to (deterministic) pushdown acceptors!* However, the converse of the above theorem is true: NDPDAs are equivalent to context-free grammars. Proofs of these facts require a considerably more extended development than we can give here, but we will include them in a final summary of the relationships between languages and machines that we have discovered in this text:

$$\text{FSA} = \text{REG} = \text{RG} = \text{NDFSA} \subsetneq \text{PDA} \subsetneq \text{CFL} = \text{NDPDA}$$

Problems

- (6.10) Trace the action of the enhanced PDA given in Example 5 with the input string $w = [[[xx]x][x[xx]]][x[x[xx]]x]$ to see if it is accepted.
- (6.11) Construct a regular (unenhanced) PDA that is equivalent to the one given in Example 5.
- (6.12) Find a sequence of possible actions of the NDPDA in Example 6 with the input string $w = aababaababaa$ which shows that it is accepted.

Bibliography

A number of very fine texts are available which give a much more extended account of the subject treated in this text. For example:

- H. R. Lewis; C. H. Papadimitriou. *Elements of the Theory of Computation*. Prentice-Hall, Upper Saddle River, N.J., 2nd Ed. (1998).
- T. A. Sudkamp. *Languages and Machines*. Addison-Wesley, 2nd Ed. (1997).

The material on finite state acceptors and their equivalences originated in several different sources:

- G. H. Mealy. “A method for synthesizing sequential circuits”. *Bell System Technical Journal* 34 (1955), 1045-1079. [invention of finite state acceptors]
- E. F. Moore. “Gedanken experiments on sequential machines”. *Automata Studies*, ed. C. E. Shannon and J. McCarthy. Princeton University Press (1956), 129-153. [algorithm for minimization]
- A. Nerode. “Linear automaton transformations”. *Proceedings of the American Mathematical Society* 9 (1958), 541-544. [Nerode-Myhill Theorem]
- M. O. Rabin; D. Scott. “Finite automata and their decision problems”. *IBM Journal of Research and Development* 3 (1959), 114-125. [equivalence of deterministic and nondeterministic FSAs]
- S. Ginsburg. “Examples of abstract machines”. *IEEE Transactions on Electronic Computers EC-11* (1962), 132-135. [finite state transducers]

The original work on CFLs and PDAs appeared in

- N. Chomsky. “Three models for the description of languages”. *IRE Transactions on Information Theory* 2 (1956), 113-124. [invention of regular and context-free grammars]
- N. Chomsky. “Context-free grammar and pushdown storage”. *Quarterly Progress Report* 65. M.I.T. Research Laboratory in Electronics, Cambridge, MA (1962), 187-194. [the equivalence]
- M. P. Schutzenberger. “On context-free languages and pushdown automata”. *Information and Control* 6 (1963), 246-264. [the equivalence]
- J. Evey. “Application of pushdown store machines”. *Proceedings of the 1963 Fall Joint Computer Conference*, Montreal: AFIPS Press (1963), 215-217. [the equivalence]